

Integer SNARKs for First-Order Logic

Murdoch J. Gabbay 

Heriot-Watt University, Edinburgh, United Kingdom

Andrew Mendelsohn 

Imperial College London, London, United Kingdom

Abstract

We efficiently arithmetise first-order logic (FOL) to SNARK-friendly relations. FOL is a highly expressive and widely-used mathematical logic capable (amongst other things) of expressing Turing-complete computational systems. We provide a framework for arithmetising FOL by extending a univariate polynomial semantics for FOL proposed by the first author (J. of Applied Logics '25) to a general semantics parameterised by a chosen tuple of functions satisfying some compatibility properties. We instantiate this framework with multilinear polynomials having integer coefficients, for their cryptographic utility. Validity judgements in the resulting semantics amount to a combination of range checks and polynomial constraints on the inputs, which fit tidily into the algebraic indexed relations of Zinc (Crypto '25). By compiling a suitable polynomial interactive oracle proof with the polynomial commitment scheme 'Zip' (cf. Zinc), we obtain a SNARK for relations expressed in FOL. Our arithmetisation runs over the integers, obviating the need for a (costly) final step of converting the initial relations into finite field-based relations.

This draft was compiled for inclusion in the zkFOL implementation repository on 29th September 2026.

2012 ACM Subject Classification Theory of computation → Interactive proof systems; Theory of computation → Cryptographic protocols; Security and privacy → Privacy-preserving protocols; Theory of computation → Logic and verification; Mathematics of computing → Probabilistic algorithms; Security and privacy → Information-theoretic techniques

Keywords and phrases first-order logic, SNARKs, cryptographic proofs, arithmetisation

Digital Object Identifier 10.4230/LIPIcs.CVIT.2016.23

Acknowledgements The authors would like to thank Albert Garreta and Alberto Centelles for helpful discussions.



© Murdoch Jamie Gabbay and Andrew Mendelsohn;
licensed under Creative Commons License CC-BY 4.0

42nd Conference on Very Important Topics (CVIT 2016).

Editors: John Q. Open and Joan R. Access; Article No. 23; pp. 23:1–23:25

Leibniz International Proceedings in Informatics



LIPICs Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl Publishing, Germany

1 Introduction

We give a compositional arithmetisation of first-order logic (FOL) to SNARK-friendly relations, using Zinc [18]. The slogan is: *cryptographic certificates of first-order validity over finite models*.

Our FOL arithmetisation generalises a prior univariate polynomial semantics [15] to a *generic* sound-and-complete semantics parameterised by tuples of functions ($\mathcal{F}$ in the mathematics below) satisfying some compatibility properties. We then instantiate this semantics with integer multilinear polynomials. This bridges from range-checked FOL validity over finite models, to polynomial constraints compatible with Zinc. Logical instance-witness pairs transform via this bridge to instance-witness pairs of a Zinc-compatible SNARK. We now explain FOL, SNARKs, and Zinc, and how they fit together.

FOL First-order logic is a powerful mathematical logic [4, 13] which more than suffices to specify computation (including Turing-complete computation) and correctness properties thereof. The following statements are all expressible in FOL: “Function f computes the square root” or “Abstract machine A , when given input x , returns output $f(x)$ ” or “Transaction t is valid according to the rules of the system” or “Proposed state-change S is compliant with legal requirement Z ”, etc. FOL is a quantified logic, with notions of universal and existential quantification: these can be expressed with the familiar symbols $\forall x.\phi(x)$ and $\exists x.\phi(x)$.

FOL comes with a well-studied and simple sets-based notion of model: we assume a set X of *elements* and we interpret universal quantification as quantification over elements in X ; existential quantification as existence of some element in X ; and we interpret an n -ary relation R as a subset of X^n ; and so on. The FOL notion of truth is simple too: a predicate is just true or false of a model, and notions of logical conjunction, disjunction, and negation are the usual Boolean operations.¹

The above properties mean we can use FOL to simply and cleanly specify finite logical and computational behaviour *and* to state correctness properties of that behaviour.² FOL, therefore, has a particularly useful application in formal verification.

However, FOL has no native cryptographic content: its intrinsic truth-theoretic notions of universal and existential quantification, $\forall x.\phi(x)$ and $\exists x.\phi(x)$, for example, simply mean ‘check $\phi(x)$ for every x ’ and ‘check possible x until we find one such that $\phi(x)$ ’.

SNARKs A SNARK is a cryptographic tool to allow a *prover* to convince a *verifier* that the prover knows a *witness* w such that $(x, w) \in \mathcal{R}$ for some *relation* $\mathcal{R}$ and *instance* x , succinctly (i.e. requiring minimal work from the verifier), and in zero knowledge (i.e. the

¹ Some shibboleths exist about FOL. “Over finite structures, NP is equal to the set of properties expressible in existential second-order logic (ESO), so FOL could not possibly express computation.” This is a category error. NP=ESO is a result in *descriptive complexity*, concerning properties of finite input structures defined by a fixed logical sentence; our use of FOL here is as a specification language whose models encode entire computations. “FOL is undecidable, so it cannot be used to do any actual reasoning.” This refers to the fact that no algorithm exists which, given an arbitrary FOL sentence ϕ , checks whether ϕ is valid in all structures. The undecidability reflects the fact that FOL theories can encode unbounded computation. However, we can check validity of a first-order formula over a particular finite structure by direct evaluation, since all quantification is over a finite domain.

² Actually, FOL is a family of logics: there is FOL with equality, FOL with function-symbols, FOL with reification, FOL with pattern-matching, and so on. We see no inherent barriers to adapting the FOL language of this paper with extra bells and whistles.

verifier does not learn information about the witness w).³

Most SNARKs work over a fixed finite field, determined by some prime power $q = p^r$ (e.g. [20, 16, 12]). Using finite field arithmetic allows instance-witness pairs in NP relations to be efficiently and succinctly checked for validity. A common three-step modular framework has been developed for constructing SNARKs:

1. Build a *polynomial interactive oracle proof* (PIOP).
2. Build a *polynomial commitment scheme* (PCS).
3. Compile the PIOP+PCS to a SNARK via the transform of [6, 8].

The PIOP is an information theoretic component, and the PCS provides cryptographic guarantees of the security of the construction.

The wrinkle. A common problem in the real-world use of SNARKs is that we often want to prove things that are not naturally expressed by relations $\mathcal{R}$ defined by means of arithmetic over finite fields, much less by arithmetic over a *single* fixed finite field.

To compensate, a so-called *arithmetisation* pre-processing step is required, which transforms the relation $\mathcal{R}$ of instance-witness pairs about which we actually want to prove knowledge, into some suitably equivalent ‘SNARK friendly’ relation $\mathcal{R}'$, which is over a finite field and thus more amenable to the technical requirements of SNARKs.

The SNARK friendly $\mathcal{R}'$ is typically defined by a polynomial expression, which may then be rewritten to an even more SNARK friendly instance of a constraint system, such as R1CS or CCS [27]. We then run the SNARK on the resulting system of constraints.

For instance: we may want to prove knowledge of the result of a computation which was performed over the rationals $\mathbb{Q}$ (which is not a finite field), or over a ring which is not a field, such as $\mathbb{Z}/n\mathbb{Z}$ for composite n . Converting these ‘natural’ relations and constraints determined by such computations into constraints over finite fields can be computationally expensive to perform, as well as causing a significant increase in the instance size upon which a SNARK is run, resulting in a costlier proof. This motivates the study of techniques to avoid or reduce the burden of arithmetisation overheads.

Zinc Zinc [18] attempts to ameliorate the cost of arithmetisation. It does not make it go away, but it pushes development effort away from the programmer and onto the cryptographic back-end. In particular, Zinc gives a PIOP for relations defined over the integers $\mathbb{Z}$, together with a PCS dubbed ‘Zip’, which compile to yield a SNARK.

Zinc allows for succinct proofs of relations with mixed characteristic, over numerous rings, to be provided by a prover. The central concept of Zinc is quite simple: for well-chosen parameters, we just sample some random primes (up to some bound) and run a finite field SNARK on the relation instances modulo the sampled primes, with low resulting soundness error. This removes the need to arithmetise such instances, beyond modular reduction. We note the relations in Zinc are defined on multilinear polynomials with integer coefficients.

Full circle back to FOL Previous work [15] shows how to translate FOL to (high degree, univariate) polynomials with integer coefficients, such that the standard FOL truth-table-and-sets-based notion of validity may be soundly and completely expressed using polynomial root tests: that is, a certain polynomial $\langle\phi\rangle$ corresponding to the predicate ϕ has roots over a

³ The abstract logical content here is minimal: the relation $\mathcal{R}$ under consideration is arbitrary. The focus here is on the cryptographic content. Even at this point, the reader can see where we are heading: we will marry the logical expressivity of FOL with the cryptographic efficiency of SNARKs, using Zinc.

certain set *if and only if* that predicate is true in its standard FOL semantics. However, [15] did not attempt to connect the resulting integer polynomials to a cryptographic backend.

In this work, therefore, we provide such a bridge to a cryptographic backend. We replace the high degree univariate FOL semantics of [15] with a multivariate semantics, instantiating the general semantics referred to in the first paragraph of this introduction with a set $\mathcal{F}$ of *multilinear* interpolating polynomials of integer witness vectors. While we could choose *any* set of functions satisfying the necessary compatibility properties for our general semantics, not only functions that are interpolant polynomials, we choose multilinear polynomials with integer coefficients, popular for their cryptographic utility in building PCS [30], for compatibility with Zinc. Validity judgements in the resulting semantics (Definition 20) then amount to a combination of range checks and polynomial constraints on the inputs (Corollary 28) which fit into the *algebraic indexed relations* of [18].

Via the polynomial interactive oracle proof (PIOP) plus polynomial commitment scheme (PCS) framework [6, 8], from any compatible PIOP and Zip [18, Section 5] we obtain a SNARK for statements expressed in FOL (Corollary 32). Our reliance on Zinc ensures that the verifier complexity of our construction is desirably low, and since Zinc is a hash-based SNARK, our construction is plausibly post-quantum secure.

1.1 Contributions

We now provide an overview of our contributions in more detail. We omit the technical details of range-checks on the various data, so that we can focus on the core ideas of the construction. We begin with a model of FOL in which we can write logical predicates; *predicate* here simply means a logical statement about an object, which can be evaluated to true or false. After defining our model of FOL, we give a *semantics* for FOL *over the integers*. This means that, at a high level, we design

1. a map (in our jargon, an *interpretation*) from the symbols comprising our model of FOL to the integers, and
2. rules for evaluating the logical veracity of FOL predicates by manipulating the integers obtained under this map (in the jargon, we can ‘evaluate validity judgments’).

The achievement of Section 2 is to design such rules which are *sound and complete*: this simply means the predicates are true (or false) *if and only if* the integer manipulations evaluate to zero (or non-zero). Our rules for integer manipulation are simple: logical equality of terms $t = t'$ is mapped to the square of the difference of the interpretations of t and t' ; logical conjunction $t \wedge t'$ (read: t AND t') is mapped to the sum of the interpretations of t and t' ; and logical disjunction $t \vee t'$ (read: t OR t') is mapped to the product of the interpretations of t and t' . We emphasise for the benefit of the reader that what is termed an ‘interpretation’ is just a map from terms to integers. The rules are defined in Figure 2 and the proof of soundness and completeness given in Theorem 19.

At this point it is necessary to provide some detail about our FOL model. We define a flavour of FOL using a novel concept we name *enriched polynomials*. To explain what these are, we introduce symbols named *matrix variable symbols*, which are pairs $(\mathbf{C}, ar(\mathbf{C}))$ consisting of a variable symbol $\mathbf{C}$ and an associated integer $ar(\mathbf{C}) \in \mathbb{N}$ (in our jargon, the ‘arity’ of $\mathbf{C}$). This associated integer should be thought of (with little harm done) as the number of ‘rows’ of the corresponding matrix variable symbol. We write the i th ‘row’ of $\mathbf{C}$ as $\mathbf{C}_i$. We define the aforementioned *interpretations* of matrix variable symbols, written $\varsigma(\mathbf{C})$, as a matrix of integers with i th row $\varsigma(\mathbf{C}_i) = \varsigma(\mathbf{C})_i$. Enriched polynomials then are a combination

of (in this paper) two or three purely formal symbols C_i , C_j , and X , suggestively presented as

$$\mathsf{C}_i(X) \quad \text{or} \quad \mathsf{C}_i\mathsf{C}_j(X)$$

$\mathsf{C}_i(X)$ reminds us of actual polynomials, and $\mathsf{C}_i\mathsf{C}_j(X)$ reminds us of composition of polynomials. We introduce these symbolic ‘polynomials’ because we can in fact map them to *actual* polynomials. To do this, we introduce a final set of variable symbols: these are symbols $B_{i,\nu}^{j,x}$ with four indices $i, j, x, \nu \in \mathbb{N}$. Here $\nu \in [\text{maxbl}]$ for some integer $\text{maxbl} \in \mathbb{N}$. We denote these by ‘ B ’ because each such symbol will correspond to a *bit* in the binary decomposition of an integer entry of $\varsigma(\mathsf{C})$. Again, we emphasise that these are just formal symbols.

To obtain polynomials we define a map inductively (see Figure 4) from enriched polynomials to multilinear polynomials over the integers. We make use of the standard polynomial which sends a string of binary digits to its corresponding integer. We denote this binary-to-integer polynomial by $b2int(X_1, \dots, X_n)$. Our inductive map is then defined by

$$\mathsf{C}_i(X) \mapsto b2int(B_{i,1}^{0,x}, \dots, B_{i,\text{maxbl}}^{0,x}), \quad \mathsf{C}_i\mathsf{C}_j(X) \mapsto b2int(B_{i,1}^{j,x}, \dots, B_{i,\text{maxbl}}^{j,x}),$$

extended homomorphically over sums and products. Thus we can take *any* FOL predicate ϕ expressed in our symbolic notation of enriched polynomials, and transform it under this map to obtain actual multivariate polynomials, constructed as sums and products of polynomials in the $B_{i,\nu}^{j,x}$ symbols. We may of course evaluate these polynomials, simply by replacing the $B_{i,\nu}^{j,x}$ symbols with bits of our choice and computing the evaluation as is done for more familiar kinds of polynomials.

The achievement of Section 3 is to prove the following remarkable property of the resulting multivariate polynomials: suppose we have a FOL predicate ϕ , defined using matrix variable symbol C , transformed to a multivariate polynomial as above using an interpretation ς , and suppose $\varsigma(\mathsf{C})$ has ℓ columns. Define the set of functions $\mathcal{F} = \{f_i = (x \in [\ell] \mapsto \varsigma(\mathsf{C})_{@i,x}) \mid i \in [\text{ar}(\mathsf{C})]\}$, where $\varsigma(\mathsf{C})_{@i,x}$ is the i, x th entry of the integer matrix $\varsigma(\mathsf{C})$. Then, if we replace

$$B_{i,\nu}^{0,x} \quad \text{with} \quad \pi_\nu \mathfrak{D}_b f_i(x), \quad \text{and} \quad B_{i,\nu}^{j,x} \quad \text{with} \quad \pi_\nu \mathfrak{D}_b f_i(f_j(x)),$$

where $\mathfrak{D}_b(x)$ is the integer-to-binary-decomposition map sending $x \in \mathbb{N}$ to a vector of bits satisfying $b2int(\mathfrak{D}_b(x)) = x$, and $\pi_\nu(v)$ is the projection onto the ν th entry of the vector v , then the evaluation of our multivariate polynomials at these specified bits is in fact *identical to the logical evaluation of ϕ* .⁴ That is to say, our multivariate polynomials evaluate at the specified bits to zero *if and only if* the FOL predicate ϕ is true (see Corollary 28).

This is the fact that will allow us to produce a SNARK for FOL in Section 4. The flow of the SNARK construction is as follows: we take a FOL predicate ϕ , generate the corresponding multivariate polynomials, denoted $mkQ_{\mathcal{F}}^x(\langle \phi \rangle)$ and parameterised over $x \in [\ell]$, and set them to be a collection of public constraints.

Morally, the witness is the interpretation $\varsigma(\mathsf{C})$. However, in order to make use of efficient polynomial commitment schemes, we compute the multilinear extensions of the rows of $\varsigma(\mathsf{C})$. This collection of polynomials takes the place of the functions $\{f_i \mid i \in [\text{ar}(\mathsf{C})]\}$ above. We then set the witness to be the vector of multilinear extension polynomials of the rows of $\varsigma(\mathsf{C})$:

$$\mathbf{w} = (f_1, \dots, f_{\text{ar}(\mathsf{C})})$$

Continuing with the the setup of our SNARK, polynomial commitments to these multilinear extensions are computed using Zip [18] and then published along with the instance data.

⁴ As long as the composition $f_i(f_j(x))$ is well-defined.

We make the mild stipulation that the outputs of the PCS commitments are represented in binary, so that composition of the multilinear extensions polynomials is well-defined.⁵ To guarantee the prover’s PCS responses are supplying binary values to the verifier, we also append to the set of instance constraints polynomials of the form

$$B_{i,\nu}^{j,x}(B_{i,\nu}^{j,x} - 1)$$

The instance constraints, polynomial commitments, and witness data are then integrated into the PIOP of Zinc, from which the SNARK is compiled and run.

We can then prove, in zero-knowledge, knowledge of tuples of binary values on which each $mkQ_{\mathcal{F}}^x(\langle\phi\rangle)$ evaluates to zero. From the foregoing exposition, these $mkQ_{\mathcal{F}}^x$ multivariate polynomials all evaluate to zero if and only if ϕ is valid, thereby proving knowledge of a witness for a *logical* instance. The resulting SNARK is analysed in Corollary 32.

A prototype (human-written) implementation of the core building blocks of our arithmetisation is at <https://anonymous.4open.science/r/zk-SNARKs-for-First-Order-Logic-2A41>. This provides a proof-of-concept, not an optimised implementation. Optimised, performant code is in active development.

Examples We showcase our work with regards to two simple examples of computation in Appendix D: exponentiation and SK combinator reduction.

We include exponentiation to illustrate that we can consider mathematical functions in our framework. It should be clear that other, more elaborate functions would just require writing other, more elaborate FOL definitions.

On the other hand, SK combinators form a powerful and mathematically convenient Turing-complete computational model. Haskell can be compiled to SK combinators directly⁶ [3] and only slightly more complex combinatory systems are used as practical compilation targets for industrial programming languages (a clean example is *Nock* [29]). Combinators admit a particularly compact expression in FOL, so the import of both examples is that we can arithmetise logical assertions (by definition; FOL is a logic), functions, *and* programs, covering (albeit in toy form) all the major food groups of logic and computation, so to speak.

Related work Other recent works have sought to address the problems of arithmetisation. Prior to Zinc, [9, 10] aimed to build a SNARK for integer relations. However, they relied on the so-called *Hidden Order Groups* assumption, which does not provide post-quantum security. There are also ‘field-agnostic’ SNARKs, which can prove validity of instances of relations defined over *any* finite field [19, 27, 7, 33]. However, these schemes could not handle the setting of infinite fields or integral domains which are not fields. There are also schemes targeting relations over finite rings which are not fields, e.g. [17, 31, 32]. In a different direction, [21, 2] designed tools (‘Distiller’, ‘Reef’) which reduce the number of constraints required to provide a proof of an original computation. Such methods could be used concurrently with our work here. Similarly [25] is concerned with reducing the amount of ‘foreign field’ arithmetic required in proof systems. Akin to our application to SK combinators, [5, 26] design SNARKs targeting computations performed in a specific programming language, in their case in C. See [22, 24] for an introduction to SNARKs.

⁵ For $f_i \circ f_j(x)$ to be well-defined, where f_i and f_j are multilinear extensions, we only require the PCS responses for queries to the commitment to f_j be returned in binary form. This observation reduces the number of variables required in our constraints and leads to a significant performance improvement.

⁶ The compilation in [3] is to SKI, but the I combinator can be expressed using S and K.

Paper structure Figure 1 gives a FOL syntax. Figure 3 gives an integer semantics $\langle \cdot \rangle_\zeta^x$. Terms evaluate to possibly negative integers, but predicates and range checks evaluate to natural numbers (Lemma 17). The integer semantics is sound and complete by Theorem 19. We then consider a chain of compilations: a compilation $\langle \cdot \rangle$ of FOL syntax to *enriched polynomials* (Figure 2); a compilation $mkQ_{\mathcal{F}}^x$ of enriched polynomials to multivariate polynomials (Figure 4); and an evaluation from multivariate polynomials to integers (Definition 25). Equivalence of this sequence of compilations with the integer semantics is our *main technical result* (Theorem 27). By Corollary 28, FOL validity is equivalent to a relation determined by polynomial equalities, so that we obtain a SNARK for FOL in Section 4.

2 Polynomials, SNARKs, FOL

Basic definitions We begin with some notation:

► **Notation 1.** (i) $\mathbb{Z} = \{\dots, -1, 0, 1, \dots\}$ is the integers and $\mathbb{N} = \{0, 1, 2, \dots\}$ is the non-negative integers and $\mathbb{Q}$ is the rational numbers. Given $n \in \mathbb{N}$ write $[n]$ for $\{1, \dots, n\}$. (ii) Write indexed NP relations as $\text{REL} = \{(i, \mathbb{x}, w)\}$. For any relation REL, let $\mathcal{L}(\text{REL}) = \{(i, \mathbb{x}) \mid \exists w : (i, \mathbb{x}, w) \in \text{REL}\}$ denote the language corresponding to REL. (iii) Write $\mathcal{R}_\partial$ for elements of a ring $\mathcal{R}$ of bit-length less than or equal to some bound $\partial > 0$. (iv) Write $\mathcal{M}_{n \times m}(\mathcal{R})$ for a matrix ring over $\mathcal{R}$ of dimension $n \times m$. (v) Write $M_{@i,j}$ for the j th element of the i th row of a matrix M . (vi) We distinguish operating symbols for logical objects from mathematical operations by using $=, <, \text{etc.}$ rather than $=, <, \text{etc.}$ for operations in FOL. (vii) Write $[\cdot]$ for an oracle to a string or polynomial within the brackets. (viii) Write π_ν for the projection taking a vector of length $\mu \geq \nu$ to its ν th element. Thus $\pi_\nu(v_1, \dots, v_\mu) = v_\nu$. (ix) Given some boolean assertion Φ (i.e. a mathematical claim that is either true or false), define $\delta(\Phi)$ the **indicator** of Φ such that $\delta(\Phi) = 0$ when Φ is true, and $\delta(\Phi) = 1$ when Φ is false. (x) We may write function application as $f(x)$ or as $f x$; meaning will always be clear.

► **Definition 2. 1.** Given a string $s = (s_1, s_2, \dots, s_\mu)$, define $b2int(s) = \sum_{1 \leq \nu \leq \mu} s_\nu \cdot 2^{\nu-1}$. In this paper, either s will be a string of binary digits (so $b2int(s)$ is an integer) or a string of variable symbols (so $b2int(s)$ is a multivariate polynomial).

2. Given $i \in \mathbb{N}$, write $\mathfrak{d}_b(i)$ for the **binary decomposition** of i .

► **Lemma 3.** For $x \in \mathbb{N}$, $\mathfrak{d}_b(x)$ is the unique string of binary digits such that $b2int(\mathfrak{d}_b(x)) = x$.

Multilinear Polynomials

► **Definition 4.** A **multilinear polynomial** over a ring $\mathcal{R}$ is a multivariate polynomial $f(\mathbf{X}) = \sum_{i=1}^{\leq \infty} a_i \prod_{j=1}^{\mu} X_j^{e_{ij}}$ in finitely many variables $\mathbf{X} = (X_1, \dots, X_\mu)$ with coefficients $a_i \in \mathcal{R}$, such that no variable occurs in any homogeneous component with multiplicity larger than one; that is, $0 \leq e_{ij} \leq 1$ for all i and for all $j \in [\mu]$. Write $\mathcal{R}^{\#}[\mathbf{X}]$ for the set of multilinear polynomials in $\mathbf{X}$ with coefficients in $\mathcal{R}$.

► **Definition 5.** Given a function $f : \{0, 1\}^\mu \rightarrow \mathcal{R}$ and a list of μ variables $\mathbf{X}$, define $\tilde{f} \in \mathcal{R}^{\#}[\mathbf{X}]$, the **multilinear extension (MLE)** of f , by

$$\tilde{f}(X_1, \dots, X_\mu) = \sum_{y \in \{0, 1\}^\mu} f(y) \cdot \text{eq}(y, \mathbf{X}), \quad \text{where} \quad \text{eq}(y, \mathbf{X}) = \prod_{i=1}^{\mu} (y_i X_i + (1 - y_i)(1 - X_i))$$

This is the unique multilinear polynomial in $\mathcal{R}^{\#}[\mathbf{X}]$ which **agrees with f on the μ -hypercube**, by which we mean that $f(y) = \tilde{f}(y)$ for every $y \in \{0, 1\}^\mu$ [28].

► **Definition 6.** Given $\mathbf{v} = (v_0, \dots, v_{2^\mu-1}) \in \mathcal{R}^{2^\mu}$, let $f_{\mathbf{v}} : \{0, 1\}^\mu \rightarrow \mathcal{R}$ take x to $f_{\mathbf{v}}(x) = v_{b2int(x)}$, where we identify x -the-binary-string with the $b2int(x)$ -th entry of $\mathbf{v}$. E.g. $f_{\mathbf{v}}(101) = v_5$. Define the **multilinear extension** of $\mathbf{v}$, $\tilde{f}_{\mathbf{v}}$, to be the multilinear extension of $f_{\mathbf{v}}$.

SNARKs and Algebraic Indexed Relations A SNARK is a complete succinct non-interactive argument of knowledge satisfying knowledge soundness (up to some error ε), allowing a prover to convince a verifier of knowledge of a witness satisfying an instance of some relation $\text{REL} = \text{REL}_{\text{gp}}$, parameterised by *global parameters* gp .

We now explain these terms in more detail. We will obtain a SNARK by composing a transformation from FOL to algebraic indexed relations (Definition 9) with Zinc-PIOP [18], and then applying the transform of [6, 8] on Zinc-PIOP with the PCS Zip [18, Section 5]. We define PIOPs, via interactive proofs, as follows:

► **Definition 7** ([18, Definition 3.2]). An **Interactive Proof (IP)** for a relation REL_{gp} with global parameters gp is a triple of algorithms (Ind, P, V) , where for all $(i, x, w) \in \text{REL}_{\text{gp}}$, Ind is a deterministic algorithm taking (gp, i) as input and outputting verifier and prover parameters $(vp, pp) \leftarrow \text{Ind}(\text{gp}, i)$. The pair $(P, V) = (P(pp, x, w), V(vp, x))$ is a pair of interactive algorithms with interaction denoted $\langle P, V \rangle$. An IP may satisfy

1. Completeness with completeness error $\varepsilon_{\text{comp}}$: for all $(i, x, w) \in \text{REL}_{\text{gp}}$,

$$\Pr [\langle P(pp, x, w), V(vp, x) \rangle = 1 \mid (vp, pp) \leftarrow \text{Ind}(\text{gp}, i)] \geq 1 - \varepsilon_{\text{comp}}(\text{gp}, i, x)$$

2. Soundness with error ε_s : for any unbounded adversarial prover P^* and any (i, x) ,

$$\Pr \left[\begin{array}{l} \langle P^*(pp, i, x), V(vp, x) \rangle = 1 \\ \wedge (i, x) \notin \mathcal{L}(\text{REL}_{\text{gp}}) \end{array} \mid (vp, pp) \leftarrow \text{Ind}(\text{gp}, i) \right] \leq \varepsilon_s(\text{gp}, i, x)$$

3. Knowledge soundness with error ε_{ks} : there exists a probabilistic extractor Ext such that, given oracle access to any unbounded adversarial prover P^* and any (i, x) ,

$$\Pr \left[\begin{array}{l} \langle P^*(pp, i, x), V(vp, x) \rangle = 1 \\ \wedge (i, x, w) \notin \text{REL}_{\text{gp}} \end{array} \mid \begin{array}{l} (vp, pp) \leftarrow \text{Ind}(\text{gp}, i) \\ w \leftarrow \text{Ext}_{P^*}(\text{gp}, i, x) \end{array} \right] \leq \varepsilon_{\text{ks}}(\text{gp}, i, x, \varepsilon_{P^*})$$

where

$$\varepsilon_{P^*} = \varepsilon_{P^*}(\text{gp}, i, x) = \Pr [\langle P^*(\text{gp}, i, x), V(vp, x) \rangle = 1]$$

This allows us to introduce (polynomial) interactive oracle proofs:

► **Definition 8** ([18, Definitions 3.3, 3.4]). An **interactive oracle proof (IOP)** is an interactive proof for an indexed relation $\text{REL}_{\text{gp}} = (i, x, w)$ in which i and x may contain oracles to strings of elements from a ring $\mathcal{R}$.

A **polynomial interactive oracle proof (PIOP)** over a ring $\mathcal{R}$ is an IOP such that all oracles contain polynomials with coefficients in $\mathcal{R}$ of prescribed number of variables μ and degrees. These polynomials can be queried at any point of $\mathcal{R}^\mu$.

Zinc-PIOP is defined for algebraic indexed relations, which as we will see comprise relations on multilinear polynomials over $\mathbb{Q}$, where coefficients are restricted to have bounded bit-length $\vartheta = \text{poly}(\lambda)$ in the security parameter λ . Here the bit-length of a rational number is computed by writing the rational number as a fraction in lowest terms, then computing the base-2 representations of the numerator and denominator (padding the smaller of the two so that they are of equal bit-size), and concatenating. This representation gives a bit-length of at most $2(\log |a| + \log |b|) + 3$, where an extra bit has been added to account for sign. Zinc's relations are defined on the rationals but honest provers are expected to use integer inputs.

We next formally define the relations used by Zinc; for the rest of this section, we fix an integral domain $\mathcal{R} \subset \mathbb{Q}$ and a tuple of global parameters $\text{gp} = (k, m, n, \mu, \partial)$ of integers defining the magnitudes and dimensions of the objects in Definition 9:

► **Definition 9.** Let $\mathcal{Q}$ be a set of multivariate polynomials with coefficients in a ring $\mathcal{R}$ and let $\mathbf{X} = (X_1, \dots, X_\mu)$ denote μ variables. An **algebraic indexed relation (AIR)** [18, Definition 4.1] is a set $\text{REL}_{\text{gp}, \mathcal{R}, \mathcal{Q}}$ of triples $(\mathbf{i}, \mathbf{x}, \mathbf{w})$ such that:

1. The **index** $\mathbf{i}$ is an n -tuple of oracles $[g_1], \dots, [g_n]$ to multilinear polynomials with ∂ -bounded coefficients, $g_1, \dots, g_n \in \mathcal{R}_\partial^\#[\mathbf{X}]$ for bit-length $\partial > 0$.
2. The **witness** $\mathbf{w}$ is a k -tuple of multilinear polynomials with ∂ -bounded coefficients, $f_1, \dots, f_k \in \mathcal{R}_\partial^\#[\mathbf{X}]$ for bit-length $\partial > 0$.
3. The **instance** $\mathbf{x} = ([f_1], \dots, [f_k], \mathbf{y})$ where $\mathbf{y} \in \mathcal{R}_\partial^m$ for some $m \geq 0$ and the $[f_i]$ are oracles to the elements in $\mathbf{w}$.
4. $\mathcal{Q}$ is a set of multivariate polynomials with coefficients in $\mathcal{R}$ in $(n+k) \cdot 2^\mu + m$ variables, each of which vanishes (i.e. is equal to zero) when evaluated on the values

$$((g_1(\mathbf{x}), \dots, g_n(\mathbf{x}), f_1(\mathbf{x}), \dots, f_k(\mathbf{x}))_{\mathbf{x} \in \{0,1\}^\mu}, \mathbf{y})$$

Summing this up in symbols, $\text{REL}_{\text{gp}, \mathcal{R}, \mathcal{Q}}$ has the following form:

$$\text{REL}_{\text{gp}, \mathcal{R}, \mathcal{Q}} = \left\{ (\mathbf{i}, \mathbf{x}, \mathbf{w}) \left| \begin{array}{l} \text{gp} = (k, m, n, \mu, \partial), \\ \mathbf{i} = ([g_1], \dots, [g_n]), \\ \mathbf{x} = ([f_1], \dots, [f_k], \mathbf{y}) \text{ for some } \mathbf{y} \in \mathcal{R}_\partial^m, \\ \mathbf{w} = (f_1, \dots, f_k) \in (\mathcal{R}_\partial^\#[\mathbf{X}])^k, \\ (g_1, \dots, g_n) \in (\mathcal{R}_\partial^\#[\mathbf{X}])^n, \\ Q((g_1(\mathbf{x}), \dots, g_n(\mathbf{x}), f_1(\mathbf{x}), \dots, f_k(\mathbf{x}))_{\mathbf{x} \in \{0,1\}^\mu}, \mathbf{y}) = 0 \\ \text{for all } Q \in \mathcal{Q} \end{array} \right. \right\} \quad (1)$$

► **Remark 10.** We can express CCS (an expressive constraint system) via Definition 9 [18, Section 4.4]. We can also express *range check/lookup* relations [27], detailed in Appendix A.

Call an index-instance-witness triple for an AIR **REL** **well-formed** when it is of the form specified in Definition 9. In particular, we assume throughout that malicious provers and adversaries use well-formed instances of AIRs, and we assume that $\mathcal{R} = \mathbb{Z}$. We thus remove the subscripted ring in our relation definitions and write $\text{REL}_{\text{gp}, \mathcal{Q}}$ for AIRs.

We refer the reader to [18, Definition 3.5] for the definition of a PCS. A multilinear PCS is **succinct** if it outputs commitments which are sublinear in 2^μ .

First-order logic We set up the syntax and denotation of a simple first-order logic:

► **Notation 11.** Here and for the rest of the paper we fix the following syntactic data:

1. A set MVS of **matrix variable symbols** $\mathbf{C} \in \text{MVS}$ with fixed arities $\text{ar}(\mathbf{C}) \geq 1$.
2. A set BVar of **quad-indexed variable symbols** $B_{i,\nu}^{j,x} \in \text{BVar}$, where $i, j, x, \nu \in \mathbb{N}$.
3. A single **index variable symbol** X .

These variable symbols are all just formal symbols. We assume they are all distinct.

The arity $\text{ar}(\mathbf{C})$ of a matrix variable symbol $\mathbf{C}$ is simply an associated positive integer. A sound and complete polynomial semantics for first-order logic (FOL) from [15] used univariate polynomials and their properties to soundly and completely capture the logical content of FOL predicates.⁷ We now construct a simplified yet expressive FOL syntax (Figure 1) and a version of the semantics in [15] (Figure 3) which are tailored to our needs in this paper.

⁷ A cryptographically oriented exposition is given in [14].

Term	$t ::= q \mid t + t \mid t * t \mid \text{len}(\mathbf{C}) \mid \text{reify}(\phi) \mid X \mid \mathbf{C}_i(X) \mid \mathbf{C}_i\mathbf{C}_j(X)$
Pred	$\phi, \psi ::= t = t \mid \phi \wedge \phi \mid \phi \vee \phi$
RCheck	$R ::= \emptyset \mid R, t < \mathbf{C}_i \mid R, \mathbf{C}_i < t$
EP	$P ::= q \mid P + P \mid P * P \mid \text{len}(\mathbf{C}) \mid X \mid \mathbf{C}_i(X) \mid \mathbf{C}_i\mathbf{C}_j(X)$ $(i, j \in [\text{ar}(\mathbf{C})], q \in \mathbb{Z})$

■ **Figure 1** Terms, predicates, range checks, and enriched polynomials (Definition 12(1))

Terms	$\langle q \rangle = q \quad (q \in \mathbb{Z})$	$\langle t + t' \rangle = \langle t \rangle + \langle t' \rangle$	$\langle \mathbf{C}_i(X) \rangle = \mathbf{C}_i(X)$
	$\langle X \rangle = X$	$\langle t * t' \rangle = \langle t \rangle * \langle t' \rangle$	$\langle \mathbf{C}_i\mathbf{C}_j(X) \rangle = \mathbf{C}_i\mathbf{C}_j(X)$
	$\langle \text{len}(\mathbf{C}) \rangle = \text{len}(\mathbf{C})$	$\langle \text{reify}(\phi) \rangle = \langle \phi \rangle$	
Predicates	$\langle t = t' \rangle = (\langle t \rangle - \langle t' \rangle)^2$	$\langle \phi \wedge \phi' \rangle = \langle \phi \rangle + \langle \phi' \rangle$	$\langle \phi \vee \phi' \rangle = \langle \phi \rangle * \langle \phi' \rangle$

■ **Figure 2** From Term and Pred to enriched polynomials EP (Definition 12(3))

- **Definition 12.** 1. Define syntaxes of *terms*, *predicates*, and *enriched polynomials* as per the BNF grammar in Figure 1.
 2. Write $\top$ as sugar for $(0 = 0)$, and $\perp$ for $(0 = 1)$.
 3. Define a mapping $\langle \cdot \rangle$ taking terms and predicates to enriched polynomials as in Figure 2.

► **Remark 13** (Some comments on Definition 12). Pred is a simple predicate language of conjunctions and disjunctions over equalities. True is expressed as $0 = 0$ and false as $0 = 1$. A quantification over the index variable symbol X is implicit in the syntax, as will become explicit later in Definition 20(3&4) and Lemma 21.

Pred only has one index variable symbol X , and one polynomial function symbol $\mathbf{C}$. In general one might want more, but even in its current form Pred is actually very expressive and not a toy: in particular, it can express the full list of examples from [15].

Enriched polynomials are a subsyntax of terms; we will exploit this fact below.

► **Lemma 14.** If $t \in \text{Term}$ and $\phi \in \text{Pred}$ then $\langle t \rangle$ and $\langle \phi \rangle$ are enriched polynomials.

Proof. By a routine induction (from Figure 2). ◀

► **Remark 15.** Why are $\langle \cdot \rangle$ and enriched polynomials interesting?

1. We can map $t \in \text{Term}$ and $\phi \in \text{Pred}$ (Figure 1) to enriched polynomials as per Lemma 14.
 2. We can map enriched polynomials to *actual* multivariate polynomials, as per Figure 4.
- Items 1 and 2 show that enriched polynomials are therefore a *bridge* between logic and multivariate polynomials. After establishing this bridge,
3. We can evaluate the variables in the resulting polynomials generated by $mkQ_{\mathcal{F}}^x$ using $\beta_{\mathcal{F}}$ (Definition 25), such that (by Theorem 27) the composition $\beta_{\mathcal{F}} \circ mkQ_{\mathcal{F}}^x \circ \langle \cdot \rangle$ coincides (by Theorem 19) with the sound and complete integer semantics for FOL $\langle \cdot \rangle_{\zeta}^x$ from Figure 3.
 4. We can use the above machinery to generate polynomial relations compatible with Zinc.

Integer Semantics for FOL In this section we define $\langle \cdot \rangle_{\zeta}^x$ (see Figure 3), describing a sound and complete integer semantics for FOL.

► **Definition 16** (FOL semantics). 1. We define an *interpretation* to be a map

$$\varsigma : \text{MVS} \rightarrow \mathcal{M}_{\text{ar}(\mathbf{C}) \times \ell_{\zeta}(\mathbf{C})}(\mathbb{N}), \quad \mathbf{C} \mapsto \varsigma(\mathbf{C})$$

$$\begin{array}{lll}
\langle q \rangle_\varsigma^x = q & \langle t + t' \rangle_\varsigma^x = \langle t \rangle_\varsigma^x + \langle t' \rangle_\varsigma^x & \langle \mathbf{C}_i(X) \rangle_\varsigma^x = \varsigma(\mathbf{C})_{@i,x} \\
\langle X \rangle_\varsigma^x = x & \langle t * t' \rangle_\varsigma^x = \langle t \rangle_\varsigma^x * \langle t' \rangle_\varsigma^x & \langle \mathbf{C}_i \mathbf{C}_j(X) \rangle_\varsigma^x = \varsigma(\mathbf{C})_{@i, \varsigma(\mathbf{C})_{@j,x}} \\
\langle \text{len}(\mathbf{C}) \rangle_\varsigma^x = \text{len}(\varsigma(\mathbf{C})) & \langle \text{reify}(\phi) \rangle_\varsigma^x = \langle \phi \rangle_\varsigma^x & \\
\langle t = t' \rangle_\varsigma^x = (\langle t \rangle_\varsigma^x - \langle t' \rangle_\varsigma^x)^2 & \langle \phi \wedge \phi' \rangle_\varsigma^x = \langle \phi \rangle_\varsigma^x + \langle \phi' \rangle_\varsigma^x & \langle \phi \vee \phi' \rangle_\varsigma^x = \langle \phi \rangle_\varsigma^x * \langle \phi' \rangle_\varsigma^x \\
\langle \emptyset \rangle_\varsigma = 0 & \langle R, \mathbf{C}_i < t \rangle_\varsigma = \langle R \rangle_\varsigma + \delta(\forall x \in [\text{len}(\varsigma(\mathbf{C}))]. \langle t \rangle_\varsigma^x > \varsigma(\mathbf{C})_{@i,x}) & \\
& \langle R, t < \mathbf{C}_i \rangle_\varsigma = \langle R \rangle_\varsigma + \delta(\forall x \in [\text{len}(\varsigma(\mathbf{C}))]. \langle t \rangle_\varsigma^x < \varsigma(\mathbf{C})_{@i,x}) &
\end{array}$$

Above: δ is the *indicator function* from Notation 1(ix), and $@_{i,x}$ is from Notation 1(v).

■ **Figure 3** Integer semantics for FOL terms, predicates, and range checks (Definition 16)

where $\ell_\varsigma(\mathbf{C}) \in \mathbb{N}_{\geq 1}$ depends on ς .⁸

2. Suppose $t \in \text{Term}$ and $\phi \in \text{Pred}$ and $R \in \text{RCheck}$ and $x \in [\ell_\varsigma(\mathbf{C})]$ and ς is an interpretation. Define a **semantics** $\langle t \rangle_\varsigma^x \in \mathbb{Z}$, $\langle \phi \rangle_\varsigma^x, \langle R \rangle_\varsigma \in \mathbb{N}$ as per Figure 3.

Definition 16 might seem odd: FOL already has a well-known Boolean truth-valued semantics, so: a) why give it an integer semantics, and b) what does this integer semantics even mean? In reply: we give FOL an integer semantics because it will help us arithmetise FOL in Section 3. As to what the semantics means, we can sum this up using two slogans:

1. *Slogan 1.* Zero = true; non-zero = false. We treat $\mathbb{N}$ as a domain of truth-values in which zero is the single designated ‘true’ truth-value, and non-zero values are ‘false’.
2. *Slogan 2.* Equality = square of difference; conjunction = sum; disjunction = product.

This just reads off the relevant clauses in Figure 3.

These slogans are mathematically justified by Lemma 17 and Theorem 19.

► **Lemma 17** (Non-negativity). Suppose $\phi \in \text{Pred}$ and ς is an interpretation. Then $\langle \phi \rangle_\varsigma^x \geq 0$.

Proof. We consider $\langle \phi \rangle_\varsigma^x$ in Figure 3: $\langle t = t' \rangle_\varsigma^x$ is a square and squares are non-negative (even if $\langle t \rangle_\varsigma^x$ or $\langle t' \rangle_\varsigma^x$ are negative); $\langle \phi \wedge \phi' \rangle_\varsigma^x$ is a sum and sums of non-negatives are non-negative; $\langle \phi \vee \phi' \rangle_\varsigma^x$ is a product and products of non-negatives are non-negative. ◀

► **Definition 18** (Validity). Suppose $\phi \in \text{Pred}$ and $R \in \text{RCheck}$ and ς is an interpretation on $\mathbf{C}$ and $x \in [\text{len}(\varsigma(\mathbf{C}))]$. Then write $x \models_\varsigma \phi$ when $\langle \phi \rangle_\varsigma^x = 0$, and write $\models_\varsigma R$ when $\langle R \rangle_\varsigma = 0$.

► **Theorem 19** (Soundness and completeness). Suppose $\phi, \phi' \in \text{Pred}$, and $t, t' \in \text{Term}$, and $R \in \text{RCheck}$. Suppose ς is an interpretation, and write $\ell = \text{len}(\varsigma(\mathbf{C}))$. Suppose $x \in [\ell]$. Then:

1. $x \models_\varsigma t = t'$ if and only if $\langle t \rangle_\varsigma^x = \langle t' \rangle_\varsigma^x$.
2. $x \models_\varsigma \top$ and $x \not\models_\varsigma \perp$ (by Definition 12(2) $\top$ is sugar for $0 = 0$ and $\perp$ is sugar for $0 = 1$).
3. $x \models_\varsigma \phi \wedge \phi'$ if and only if $x \models_\varsigma \phi \wedge x \models_\varsigma \phi'$.
4. $x \models_\varsigma \phi \vee \phi'$ if and only if $x \models_\varsigma \phi \vee x \models_\varsigma \phi'$.
5. $\models_\varsigma \emptyset$.
6. $\models_\varsigma R, t < \mathbf{C}_i$ if and only if $\models_\varsigma R \wedge \forall x \in [\ell]. \langle t \rangle_\varsigma^x < \varsigma(\mathbf{C})_{@i,x}$.
7. $\models_\varsigma R, \mathbf{C}_i < t$ if and only if $\models_\varsigma R \wedge \forall x \in [\ell]. \varsigma(\mathbf{C})_{@i,x} < \langle t \rangle_\varsigma^x$.

Proof. See Appendix B. ◀

⁸ Letting $\varsigma(\mathbf{C})$ be a matrix of *non-negative* numbers is convenient because it simplifies *b2int* and $\mathfrak{b}_b$ in Definition 2 (there is no need to worry about binary representations of negative numbers). We can still get negative numbers in terms t using addition and multiplication by $q \in \mathbb{Z}$.

$$\begin{aligned}
mkQ_{\mathcal{F}}^x(q) &= q & mkQ_{\mathcal{F}}^x(P + P') &= mkQ_{\mathcal{F}}^x(P) + mkQ_{\mathcal{F}}^x(P') \\
mkQ_{\mathcal{F}}^x(\text{len}(\mathbf{C})) &= \ell & mkQ_{\mathcal{F}}^x(P * P') &= mkQ_{\mathcal{F}}^x(P) * mkQ_{\mathcal{F}}^x(P') \\
mkQ_{\mathcal{F}}^x(X) &= x & mkQ_{\mathcal{F}}^x(\mathbf{C}_i(X)) &= b2int(B_{i,1}^{0,x}, \dots, B_{i,maxbl}^{0,x}) \\
& & mkQ_{\mathcal{F}}^x(\mathbf{C}_i\mathbf{C}_j(X)) &= b2int(B_{i,1}^{j,x}, \dots, B_{i,maxbl}^{j,x})
\end{aligned}$$

Here $i \in [ar(\mathbf{C})]$, $j \in \{0\} \cup [ar(\mathbf{C})]$, $maxbl, \ell \in \mathbb{N}_{\geq 1}$, $x \in [\ell]$, and $\mathcal{F} = (f_i : [\ell] \rightarrow \mathbb{N} \mid i \in [ar(\mathbf{C})])$. Note that $j \in \{0\} \cup [ar(\mathbf{C})]$ yet $\mathbf{C}_0(X)$ is not defined - this will not trouble us because a FOL validity predicate will never transform to an enriched polynomial supported on $\mathbf{C}_0(X)$.

■ **Figure 4** From enriched polynomials EP to multivariate polynomials MVP (Definition 24)

3 Arithmetising FOL

We now show how to arithmetise the FOL syntax from Figure 1, i.e. to map FOL to polynomials that match up, in a sense made formal by Theorem 27, with the integer semantics of Theorem 19.

- **Definition 20.** 1. Say $\phi \in \text{Pred}$ **uses $\mathbf{C}_j$ as a pointer** when ϕ contains a subterm of the form $\mathbf{C}_i\mathbf{C}_j(X)$.
2. Say $(\phi, R) \in \text{Pred} \times \text{RCheck}$ is **range-checked** when for every j such that ϕ uses $\mathbf{C}_j$ as a pointer, R contains conditions of the form $0 < \mathbf{C}_j$ and $\mathbf{C}_j < \text{len}(\mathbf{C}) + 1$.
3. A **judgement** is a 3-tuple, written $\mathbf{C} \models_{\varsigma} \phi; R$, of: an interpretation ς ; a predicate $\phi \in \text{Pred}$; and a range check $R \in \text{RCheck}$; such that (ϕ, R) is range-checked.
4. Call a judgement $\mathbf{C} \models_{\varsigma} \phi; R$ **valid** when: $\langle \phi \rangle_{\varsigma}^x = 0$ for every $x \in [\text{len}(\varsigma(\mathbf{C}))]$, and $\langle R \rangle_{\varsigma} = 0$.
5. We may write $\mathbf{C} \models_{\varsigma} \phi; \emptyset$ just as $\mathbf{C} \models_{\varsigma} \phi$, and we may write $\mathbf{C} \models_{\varsigma} \mathbf{T}; R$ just as $\mathbf{C} \models_{\varsigma} R$.

► **Lemma 21.** Let $\phi \in \text{Pred}$, $R \in \text{RCheck}$, ς be an interpretation, and $\ell = \text{len}(\varsigma(\mathbf{C}))$. Then:

1. $\mathbf{C} \models_{\varsigma} \phi$ when $\langle \phi \rangle_{\varsigma}^x = 0$ (equivalently using Definition 18: $x \models_{\varsigma} \phi$) for every $x \in [\ell]$.
Note $\mathbf{C} \models_{\varsigma} \phi$ has the flavour of a universal quantification of ϕ for X ranging over $x \in [\ell]$.
2. $\mathbf{C} \models_{\varsigma} R$ when $\langle R \rangle_{\varsigma} = 0$ (equivalently: $\models_{\varsigma} R$).
We unpack what this means using Theorem 19(6&7): for every $x \in [\ell]$ we have $\langle t \rangle_{\varsigma}^x < \varsigma(\mathbf{C})_{@i,x}$ for every $t < \mathbf{C}_i$ in R , and $\langle t \rangle_{\varsigma}^x > \varsigma(\mathbf{C})_{@i,x}$ for every $\mathbf{C}_i < t$ appearing in R .

Proof. Direct from Definition 20. ◀

► **Remark 22.** As per Lemma 21, $\mathbf{C} \models_{\varsigma} \phi; R$ being valid has the flavour of a universal quantification that ϕ and R are valid for every $x \in [\ell]$. This gives our syntax from Figure 1 some extra expressivity, in that the index variable X is universally quantified over $[\ell]$.

► **Remark 23.** We continue Remark 15: predicates $\phi \in \text{Pred}$ (Figure 1) can express equality, conjunction, and disjunction on terms that include $\mathbf{C}_i(X)$ and $\mathbf{C}_i\mathbf{C}_j(X)$. Range checks $R \in \text{RCheck}$ can express range checks on the $\mathbf{C}_i$.

The integer semantics from Figure 3 maps this syntax to integers, and Theorem 19 shows that this fits together in the sense that with these definitions, validity of predicates and range checks behaves in the way that the symbols would lead us to expect. For example: $x \models_{\varsigma} \phi \wedge \phi'$ is indeed valid if and only if $x \models_{\varsigma} \phi$ is valid and $x \models_{\varsigma} \phi'$ is valid; and $t < \mathbf{C}_i$ is indeed valid when $\langle t \rangle_{\varsigma}^x$ is a lower bound for the entries of the i th row of $\varsigma(\mathbf{C})$.

As per Remark 15, we will now connect our integer semantics to a multivariate polynomial semantics which we build using the enriched polynomials from Figure 2.

We now define mkQ (read ‘make Q ’) taking enriched polynomials to multivariate polynomials. Intuitively, $mkQ_{\mathcal{F}}^x$ is what computes the polynomials $Q \in \mathcal{Q}$ used in equation (1).

We pad binary expansions of the elements of the images of the $f_i \in \mathcal{F}$ so that all binary expansions have the same length, equal to the largest bit-length obtaining over all the ranges of the f_i . Write $maxbl$ for the *maximum bit-length* of the elements in the images of the f_i .

► **Definition 24.** Suppose $maxbl, \ell > 0$, $x \in [\ell]$, and $\mathcal{F} = (f_i : [\ell] \rightarrow \mathbb{N} \mid i \in [ar(\mathcal{C})])$. Define $mkQ_{\mathcal{F}}^x$ mapping enriched polynomials (Definition 12(1)) to multivariate polynomials

$$mkQ_{\mathcal{F}}^x : \text{EP} \rightarrow \mathbb{Z}[B_{1,1}^{0,1}, \dots, B_{1,maxbl}^{0,1}, \dots, B_{ar(\mathcal{C}),1}^{ar(\mathcal{C}),\ell}, \dots, B_{ar(\mathcal{C}),maxbl}^{ar(\mathcal{C}),\ell}]$$

as per Figure 4.

Next we define an evaluation map $\beta_{\mathcal{F}}$ as a composition of the π_{ν} , $\mathfrak{D}_{\mathbf{b}}$, and the f_i :

► **Definition 25.** Suppose $maxbl, \ell > 0$ and $\mathcal{F} = (f_i : [\ell] \rightarrow \mathbb{N} \mid i \in [ar(\mathcal{C})])$.⁹ Then:

1. For each $i \in [ar(\mathcal{C})]$ and $j \in [ar(\mathcal{C})]$ and $x \in [\ell]$ and $\nu \in [maxbl]$, define $\beta_{\mathcal{F}}$ by

$$\beta_{\mathcal{F}}(B_{i,\nu}^{0,x}) = \pi_{\nu} \mathfrak{D}_{\mathbf{b}} f_i(x) \quad \text{and} \quad \beta_{\mathcal{F}}(B_{i,\nu}^{j,x}) = \begin{cases} \pi_{\nu} \mathfrak{D}_{\mathbf{b}} f_i(f_j(x)) & \text{if } f_j(x) \in [\ell] \\ 0 & \text{if } f_j(x) \notin [\ell] \end{cases}$$

Thus in words: $\beta_{\mathcal{F}}$ maps $B_{i,\nu}^{0,x}$ to the ν th bit of the binary expansion of $f_i(x)$; and $\beta_{\mathcal{F}}$ maps $B_{i,\nu}^{j,x}$ to the ν th bit of the binary expansion of $f_i(f_j(x))$ (if this is in-range, Definition 26).

2. We extend $\beta_{\mathcal{F}}$ to an **evaluation map** on multivariate polynomials in the natural way by instantiating variable symbols. E.g. $\beta_{\mathcal{F}}(0 + B_{1,3}^{0,2} * B_{4,6}^{0,5}) = 0 + (\pi_3 \mathfrak{D}_{\mathbf{b}} f_1(2)) * (\pi_6 \mathfrak{D}_{\mathbf{b}} f_4(5))$.

► **Definition 26.** Suppose $\mathcal{F} = (f_i : [\ell] \rightarrow \mathbb{N} \mid i \in [ar(\mathcal{C})])$ and $t \in \text{Term}$ and $\phi \in \text{Pred}$. Call $\mathcal{F}$ **in-range** for t / for ϕ when for every j that t / ϕ uses as a pointer, we have $\forall x \in [\ell]. 1 \leq f_j(x) \leq \ell$.

A central technical result is that $\beta_{\mathcal{F}} \circ mkQ_{\mathcal{F}}^x \circ \langle \cdot \rangle$ equals $\langle \cdot \rangle_{\varsigma}^x$:

► **Theorem 27.** Suppose $t \in \text{Term}$ and $\phi \in \text{Pred}$ and ς is an interpretation. Write $\ell = \text{len}(\varsigma(\mathcal{C}))$ and suppose $x \in [\ell]$. Define $\mathcal{F} = (f_i = (x \in [\ell] \mapsto \varsigma(\mathcal{C})_{@i,x}) \mid i \in [ar(\mathcal{C})])$. If $\mathcal{F}$ is in-range for t and ϕ (Definition 26), then:

$$\beta_{\mathcal{F}}(mkQ_{\mathcal{F}}^x \langle t \rangle) = \langle t \rangle_{\varsigma}^x \quad \text{and} \quad \beta_{\mathcal{F}}(mkQ_{\mathcal{F}}^x \langle \phi \rangle) = \langle \phi \rangle_{\varsigma}^x$$

Proof. Routine induction on syntax. The interesting cases are for X , $\mathbf{C}_i(X)$, and $\mathbf{C}_i \mathbf{C}_j(X)$:

$$\begin{aligned} \beta_{\mathcal{F}}(mkQ_{\mathcal{F}}^x \langle X \rangle) &= \beta_{\mathcal{F}}(mkQ_{\mathcal{F}}^x X) && \text{Figure 2} \\ &= \beta_{\mathcal{F}}(x) && \text{Figure 4} \\ &= x && \text{Def. 25(2)} \\ &= \langle X \rangle_{\varsigma}^x && \text{Figure 3} \\ \beta_{\mathcal{F}}(mkQ_{\mathcal{F}}^x \langle \mathbf{C}_i(X) \rangle) &= \beta_{\mathcal{F}}(b2int(B_{i,1}^{0,x}, \dots, B_{i,maxbl}^{0,x})) && \text{Figures 2 \& 4} \\ &= b2int(\mathfrak{D}_{\mathbf{b}} f_i(x)) && \text{Def. 25} \\ &= f_i(x) && \text{Lemma 3} \\ &= \varsigma(\mathcal{C})_{@i,x} && \text{Def. of } \mathcal{F}, x \in [\ell] \\ &= \langle \mathbf{C}_i(X) \rangle_{\varsigma}^x && \text{Figure 3} \\ \beta_{\mathcal{F}}(mkQ_{\mathcal{F}}^x \langle \mathbf{C}_i \mathbf{C}_j(X) \rangle) &= \beta_{\mathcal{F}}(b2int(B_{i,1}^{j,x}, \dots, B_{i,maxbl}^{j,x})) && \text{Figures 2 \& 4} \\ &= f_i(f_j(x)) && \text{Def. 25 \& Lemma 3} \\ &= \varsigma(\mathcal{C})_{@i,\varsigma(\mathcal{C})_{@j,x}} && \text{Def. of } \mathcal{F}, f_j(x) \in [\ell] \\ &= \langle \mathbf{C}_i \mathbf{C}_j(X) \rangle_{\varsigma}^x && \text{Figure 3} \end{aligned}$$

⁹ In the case of a cryptographic application to arithmetise FOL relations to AIRs, a verifier V will not have direct access to the f_i , instead having oracle access to them via a polynomial commitment scheme.

► **Corollary 28.** *Suppose ς is an interpretation such that $\varsigma(\mathbf{C}) \in \mathcal{M}_{ar(\mathbf{C}) \times \ell}(\mathbb{N})$. Suppose $\mathbf{C} \models \phi; R$ is a judgement (so that $\phi \in \text{Pred}$ and $R \in \text{RCheck}$ and (ϕ, R) is properly range-checked). Define $\mathcal{F} = (f_i = (x \in [\ell] \mapsto \varsigma(\mathbf{C})_{@i,x}) \mid i \in [ar(\mathbf{C})])$. Then:*

$$\begin{aligned} \mathbf{C} \models_{\varsigma} \phi; R \quad \text{if and only if} \quad & \forall x \in [\ell]. \beta_{\mathcal{F}}(mkQ_{\mathcal{F}}^x\langle\phi\rangle) = 0 \quad \wedge \\ & \forall x \in [\ell]. \beta_{\mathcal{F}}(mkQ_{\mathcal{F}}^x\langle t \rangle) < \varsigma(\mathbf{C})_{@i,x} \text{ for every } t < \mathbf{C}_i \text{ in } R \quad \wedge \\ & \forall x \in [\ell]. \beta_{\mathcal{F}}(mkQ_{\mathcal{F}}^x\langle t \rangle) > \varsigma(\mathbf{C})_{@i,x} \text{ for every } \mathbf{C}_i < t \text{ in } R \end{aligned}$$

Proof. Routine from Theorem 27 and Lemma 21. Theorem 27 requires $\mathcal{F}$ be in-range (Definition 26); this follows from our assumption that (ϕ, R) is range-checked (Definition 20(2)). ◀

4 An integer SNARK for Relations in First-Order Logic

Below, we describe in Lemma 30 how to transform instances of relations defined in first-order logic to instances of relations defined using multivariate polynomials, using Corollary 28. These latter relations are suitable inputs to Zinc-PIOP. We do not require auxiliary data in the index $\mathbf{i}$, so we only consider instance-witness pairs $(\mathbf{x}, \mathbf{w})$ below. We first define relations for predicates in Pred and range checks in RCheck :

► **Definition 29.** *Suppose $\phi \in \text{Pred}$ (the predicate language from Figure 1), and $R \in \text{RCheck}$, and ς is an interpretation. Let $\ell = \text{len}(\varsigma(\mathbf{C}))$. Define two relations:*

$$\begin{aligned} \text{REL}_{\text{gp}}^{\text{FOL}} &= \{ ((\phi, R), \varsigma) \mid \varsigma(\mathbf{C}) \in \mathcal{M}_{ar(\mathbf{C}) \times \ell}(\mathbb{N}), \quad \mathbf{C} \models_{\varsigma} \phi; R \} \quad \text{and} \\ \text{REL}_{\text{gp}}^{\text{EP}} &= \{ ((\langle\phi\rangle, R), \varsigma(\mathbf{C})) \mid \varsigma(\mathbf{C}) \in \mathcal{M}_{ar(\mathbf{C}) \times \ell}(\mathbb{N}), \quad \mathbf{C} \models_{\varsigma} \phi; R \} \end{aligned}$$

The relation $\text{REL}_{\text{gp}}^{\text{EP}}$ is an intermediate relation obtained by turning instances of $\text{REL}_{\text{gp}}^{\text{FOL}}$ into enriched polynomials, via Figure 2. By converting ϕ to $\langle\phi\rangle$, we obtain an enriched polynomial instance preserving the logical content of ϕ , with a witness $\mathbf{w} = \varsigma$ being knowledge of $\varsigma(\mathbf{C}) \in \mathcal{M}_{ar(\mathbf{C}) \times \ell}(\mathbb{N})$ satisfying ϕ .

We now use $mkQ_{\mathcal{F}}^x$ (Figure 4) to map enriched polynomials to multivariate polynomials, setting $\mathcal{F} = (\tilde{f}_{\varsigma(\mathbf{C})}^i \mid i \in [ar(\mathbf{C})])$, i.e. $\mathcal{F}$ comprises the multilinear extensions of the rows of $\varsigma(\mathbf{C})$. Note that, before padding, the $\tilde{f}_{\varsigma(\mathbf{C})}^i$ are multilinear polynomials in $\mu = \lceil \log_2 \ell \rceil$ variables. However, for pointer correctness, we pad so that $\mu = \max(\lceil \log_2 \ell \rceil, \text{maxbl})$.¹⁰

By Corollary 28, with $\mathcal{F} = (\tilde{f}_{\varsigma(\mathbf{C})}^i \mid i \in [ar(\mathbf{C})])$ and $\tilde{f}_{\varsigma(\mathbf{C})}^i(\tilde{f}_{\varsigma(\mathbf{C})}^j(x)) := \tilde{f}_{\varsigma(\mathbf{C})}^i(\mathbb{0}_b \tilde{f}_{\varsigma(\mathbf{C})}^j(x))$, we have that $\text{REL}_{\text{gp}}^{\text{EP}}$ of Definition 29 is equivalent to

$$\text{REL}_{\text{gp}, \mathcal{F}}^{\text{MV}} = \left\{ \left(((mkQ_{\mathcal{F}}^x\langle\phi\rangle)_{x \in [\ell]}, R), (\tilde{f}_{\varsigma(\mathbf{C})}^i)_{i \in [ar(\mathbf{C})]} \right) \mid \begin{array}{l} \varsigma(\mathbf{C}) \in \mathcal{M}_{ar(\mathbf{C}) \times \ell}(\mathbb{N}), \quad \mathbf{C} \models_{\varsigma} R \\ \forall x \in [\ell] (\beta_{\mathcal{F}}(mkQ_{\mathcal{F}}^x\langle\phi\rangle) = 0) \end{array} \right\}$$

By choosing $\mathcal{F} = \{\tilde{f}_{\varsigma(\mathbf{C})}^i \mid i \in [ar(\mathbf{C})]\}$, we find that the condition that the image of $\beta_{\mathcal{F}}$ on the multivariate polynomial $mkQ_{\mathcal{F}}^x\langle\phi\rangle$ satisfies $\beta_{\mathcal{F}}(mkQ_{\mathcal{F}}^x\langle\phi\rangle) = 0$ is equivalent to $mkQ_{\mathcal{F}}^x\langle\phi\rangle$ evaluating to zero at some subset of

$$(\mathbb{0}_b(\varsigma(\mathbf{C})_{@1,1}), \dots, \mathbb{0}_b(\varsigma(\mathbf{C})_{@ar(\mathbf{C}), \ell})) = (\mathbb{0}_b \tilde{f}_{\varsigma(\mathbf{C})}^1(\mathbb{0}_b 1), \dots, \mathbb{0}_b \tilde{f}_{\varsigma(\mathbf{C})}^{ar(\mathbf{C})}(\mathbb{0}_b \ell))$$

¹⁰ More explicitly, the $\tilde{f}_{\varsigma(\mathbf{C})}^i$ have domain $[\text{len}(\varsigma(\mathbf{C}))]$, so inputs are of bit length at most $\lceil \log_2(\text{len}(\varsigma(\mathbf{C}))) \rceil = \mu$, and have range comprising elements of bit length at most maxbl . To form the composition $\tilde{f}_{\varsigma(\mathbf{C})}^i(\mathbb{0}_b \tilde{f}_{\varsigma(\mathbf{C})}^j)$, we require that the length of $\mathbb{0}_b \tilde{f}_{\varsigma(\mathbf{C})}^j$ is identical to the bit length of inputs to $\tilde{f}_{\varsigma(\mathbf{C})}^i$.

(recall from Notation 1(v) that $\varsigma(\mathbf{C})_{@i,j}$ is the i, j th entry of $\varsigma(\mathbf{C})$) as long as the range checks in R guarantee that any pointer $\mathbf{C}_i(X)$ in ϕ corresponds to a row $\varsigma(\mathbf{C})_i$ with entries bounded by $\text{len}(\varsigma(\mathbf{C}))$. We now prove that $\text{REL}_{\text{gp}, \mathcal{F}}^{\text{MV}}$ provides algebraic indexed relations:

► **Lemma 30.** *Let gp_0, gp_1 denote global parameters, and fix $\partial, \mu > 0$. Let ϕ be a predicate defined over $\mathbb{Z}$ in a matrix variable symbol $\mathbf{C}$. Let $\varsigma(\mathbf{C}) \in \mathcal{M}_{\text{ar}(\mathbf{C}) \times \ell}(\mathbb{N})$ be an interpretation, and R be a set of range check conditions on $\varsigma(\mathbf{C})$. Set $\mathcal{F} = \{\tilde{f}_{\varsigma(\mathbf{C})}^i \in \mathbb{Z}_{\partial}^{\text{ll}}[\mathbf{X}] \mid i \in [\text{ar}(\mathbf{C})]\}$. Then the data of an instance of $\text{REL}_{\text{gp}_0}^{\text{EP}}$ can be expressed as a pair $(\mathbb{x}, \mathbb{w})$ of $\text{REL}_{\text{gp}_1, \mathcal{Q}}$ for some gp_1 and some $\mathcal{Q}$, such that $(\mathbb{x}, \mathbb{w})$ is well-formed if and only if $\mathbf{C} \models_{\varsigma} \phi$; R is valid.*

Proof. Let

$$(((mkQ_{\mathcal{F}}^x(\phi))_{x \in [\text{len}(\varsigma(\mathbf{C}))]}, R), (\tilde{f}_{\varsigma(\mathbf{C})}^i)_{i \in [\text{ar}(\mathbf{C})]}) \in \text{REL}_{\text{gp}, \mathcal{F}}^{\text{MV}}$$

be obtained from the instance $((\phi, R), \varsigma) \in \text{REL}_{\text{gp}_0}^{\text{EP}}$. We first define an AIR, $\text{REL}_{\text{gp}', \mathcal{Q}'}$, corresponding to $(mkQ_{\mathcal{F}}^x(\phi))_{x \in [\text{len}(\varsigma(\mathbf{C}))]}$. Set the instance-witness pair in $\text{REL}_{\text{gp}', \mathcal{Q}'}$ to be

$$(\mathbb{x}, \mathbb{w}) = ([\tilde{f}_{\varsigma(\mathbf{C})}^1], \dots, [\tilde{f}_{\varsigma(\mathbf{C})}^{\text{ar}(\mathbf{C})}], (\tilde{f}_{\varsigma(\mathbf{C})}^1, \dots, \tilde{f}_{\varsigma(\mathbf{C})}^{\text{ar}(\mathbf{C})}))$$

where $[\cdot]$ denotes an oracle to the argument,¹¹ and set

$$\mathcal{Q}' = (mkQ_{\mathcal{F}}^x(\phi))_{x \in [\text{len}(\varsigma(\mathbf{C}))]}$$

These data represent an AIR for some gp' : for each $x \in [\text{len}(\varsigma(\mathbf{C}))]$, $mkQ_{\mathcal{F}}^x(\phi)$ is a multivariate polynomial determined by ϕ , evaluating to zero on a subset of the binary decompositions

$$(\mathfrak{O}_b \tilde{f}_{\varsigma(\mathbf{C})}^1(\mathfrak{O}_b 1), \dots, \mathfrak{O}_b \tilde{f}_{\varsigma(\mathbf{C})}^{\text{ar}(\mathbf{C})}(\mathfrak{O}_b \text{len}(\varsigma(\mathbf{C}))))$$

We also append polynomials of the form $B_{i,1}^{j,x}(B_{i,1}^{j,x} - 1), \dots, B_{i,\mu}^{j,x}(B_{i,\mu}^{j,x} - 1)$ to $\mathcal{Q}'$; for each response from the oracle the verifier will check the vector has length μ and use the $B_{i,\nu}^{j,x}(B_{i,\nu}^{j,x} - 1)$ to ensure that the oracle has returned data of the correct data type, namely bits.

Next, since a range check may be encoded as an AIR (equation (2)), for all range check conditions in R we form a corresponding AIR with a set of defining polynomials $\mathcal{Q}_R$ and global parameters gp'' . We then combine the two AIRs we have constructed, setting $\mathcal{Q} := \mathcal{Q}' \cup \mathcal{Q}_R$, appending the instance-witness data of the range check relation to the former relation $\text{REL}_{\text{gp}', \mathcal{Q}'}$, and denoting the updated global parameters by gp_1 .

The claim that $(\mathbb{x}, \mathbb{w})$ is well-formed if and only if $\mathbf{C} \models_{\varsigma} \phi$; R is direct from Corollary 28: well-formed $(\mathbb{x}, \mathbb{w})$ have integer entries and satisfy the polynomial relations in $\mathcal{Q}$, whose constraints guarantee that $\mathbf{C} \models_{\varsigma} \phi$ and $\mathbf{C} \models_{\varsigma} R$. The converse holds by construction. ◀

For instances $((\phi, R), \varsigma)$ to generate valid inputs to Zinc, they must also be *well-formed*:

► **Definition 31.** *An instance $((\phi, R), \varsigma) \in \text{REL}_{\text{gp}}^{\text{FOL}}$ is **well-formed** if it transforms via the transform described in Lemma 30 to an instance of $\text{REL}_{\text{gp}, \mathcal{Q}}$, for some $\mathcal{F}$ and $\mathcal{Q}$, which is well-formed as an algebraic indexed relation.*

Lemma 30 shows that we may efficiently compile well-formed instance-witness pairs defined in the syntax of FOL (matrix variable symbols and enriched polynomials) into AIR instance-witness pairs which are valid inputs to Zinc-PIOP. Once we have instance-witness pairs of a form which may be provided as the input to a PIOP (such as Zinc-PIOP), we can commit to the witness data using a PCS and compile the PIOP to obtain a SNARK.

¹¹ Which oracle will later be instantiated with a PCS such as Zip [18, Section 5].

Observe from Figure 4 that $mkQ_{\mathcal{F}}^x\langle\phi\rangle$ may contain subterms corresponding to $C_iC_j(X)$, whose indices depend on evaluations $f_j(x) = \tilde{f}_{\varsigma(\mathbb{C})}^j(\mathfrak{b}_b x)$. Recall further that such $\tilde{f}_{\varsigma(\mathbb{C})}^j$ are ‘pointers’ obtained by interpolating rows of $\varsigma(\mathbb{C})$. Since these values (equivalently, interpolating polynomials) are not public, but rather are part of the witness inside a polynomial commitment, when running a PIOP on the relations constructed in Lemma 30 we stipulate that a verifier who verifies that $\beta_{\mathcal{F}}(mkQ_{\mathcal{F}}^x\langle\phi\rangle) = 0$ must first query the commitments $[\tilde{f}_{\varsigma(\mathbb{C})}^j(X_1, \dots, X_{\mu})] \in \mathbb{X}$ for $\mathfrak{b}_b \tilde{f}_{\varsigma(\mathbb{C})}^j(\mathfrak{b}_b x)$ for all j for which $\varsigma(\mathbb{C})_j$ acts as a pointer, which values then define $mkQ_{\mathcal{F}}^x\langle\phi\rangle$. We assume the PCS returns the bits of the queried entry of the witness data. Aside from this additional step, our SNARK uses a multilinear PCS to provide input relations for a PIOP which is run in an otherwise standard manner.

► **Corollary 32.** *Let λ denote a security parameter. Suppose $\phi \in \text{Pred}$ and $R \in \text{RCheck}$ and ς is an interpretation satisfying $\varsigma(\mathbb{C}) \in \mathcal{M}_{ar(\mathbb{C}) \times len(\varsigma(\mathbb{C}))}(\mathbb{N})$, with $ar(\mathbb{C}), len(\varsigma(\mathbb{C})) \in \mathbb{Z}_{\geq 1}$ satisfying $ar(\mathbb{C}) = \text{poly}(\lambda)$, $len(\varsigma(\mathbb{C})) = \text{poly}(\lambda)$. Set $\mu := \text{maxbl}$. Set $\mathcal{F} = \{\tilde{f}_{\varsigma(\mathbb{C})}^i \in \mathbb{Z}_{\partial}^{\mathbb{N}}[\mathbf{X}] \mid i \in [ar(\mathbb{C})]\}$ for some $\partial > 0$. Then there exists a SNARK to convince a verifier that a well-formed instance $((\phi, R), \varsigma) \in \text{REL}_{\text{gp}}^{\text{FOL}}$ satisfies $\mathbb{C} \models_{\varsigma} \phi; R$, with soundness error $\varepsilon_{\text{sound}}$ equal to the soundness error of Zinc, instantiated with global parameters $\text{gp} = (ar(\mathbb{C}), 0, 1, (|R| + 2)\mu, \partial)$.*

Proof. See Appendix C. ◀

► **Remark 33.** The verifier complexity of our SNARK is inherited from the verifier complexity of the SNARK used under the hood of Zinc. The verifier sees a proof, which they verify. The prover complexity is more complicated. If the input instance is defined on the $len(\varsigma(\mathbb{C}))$ polynomials $(mkQ_{\mathcal{F}}^x\langle\phi\rangle)_{x \in [len(\varsigma(\mathbb{C}))]}$, then the prover complexity is also directly inherited from the prover complexity of Zinc’s SNARK.

However, if the input instance is defined from an arbitrary FOL predicate, then the prover must perform the transformations of Figures 2 and 4 to obtain the $mkQ_{\mathcal{F}}^x\langle\phi\rangle$. For an arbitrary predicate ϕ this may be computationally intensive. However, in practice provers do not generate arbitrary ϕ ; they generate ϕ with specific structure reflecting specific meaning. Thus there is a design space of optimal syntactic forms for predicates, to optimise proving. This is a familiar pattern: design spaces of optimal syntactic forms are known for other computational applications of logic (e.g. the *Horn clause theories* of logic-programming). We leave as an open question the task of preprocessing the input FOL relations into a logically equivalent form which arithmetises to an AIR of lowest complexity.

► **Remark 34.** We note that the set $\mathcal{Q}'$ contains $len(\varsigma(\mathbb{C}))$ polynomials $mkQ_{\mathcal{F}}^x\langle\phi\rangle$. Moreover, the equivalence between the initial FOL instance and the instance we construct as an input to Zinc-PIOP holds only if $\beta_{\mathcal{F}}(mkQ_{\mathcal{F}}^x\langle\phi\rangle) = 0$ for all $x \in [len(\varsigma(\mathbb{C}))]$. To avoid checking that each and every $mkQ_{\mathcal{F}}^x\langle\phi\rangle$ evaluates to zero under $\beta_{\mathcal{F}}$, we can run a ‘random linear combinations’ step in which the verifier sends $\ell_{\varsigma}(\mathbb{C}) = len(\varsigma(\mathbb{C}))$ uniformly random challenges c_i from a suitable challenge set and the prover proves that the random linear combination

$$\sum_{x=1}^{\ell_{\varsigma}(\mathbb{C})} c_x \cdot mkQ_{\mathcal{F}}^x\langle\phi\rangle \quad \text{satisfies} \quad \beta_{\mathcal{F}} \left(\sum_{x=1}^{\ell_{\varsigma}(\mathbb{C})} c_x \cdot mkQ_{\mathcal{F}}^x\langle\phi\rangle \right) = 0$$

This comes at the cost of a small increase in the soundness error of the protocol, and the details are beyond the scope of this paper.

References

- 1 Logan Allen, Brian Klatt, Philip Quirk, and Yaseen Shaikh. EDEN - a practical, SNARK-friendly combinator VM and ISA. Cryptology ePrint Archive, Paper 2023/1021, 2023. URL: <https://eprint.iacr.org/2023/1021>.
- 2 Sebastian Angel, Eleftherios Ioannidis, Elizabeth Margolin, Srinath Setty, and Jess Woods. Reef: Fast succinct Non-Interactive Zero-Knowledge regex proofs. In *33rd USENIX Security Symposium (USENIX Security 24)*, pages 3801–3818, Philadelphia, PA, August 2024. USENIX Association. URL: <https://www.usenix.org/conference/usenixsecurity24/presentation/angel>.
- 3 Lennart Augustsson. MicroHs: A small compiler for Haskell. In *Proceedings of the 17th ACM SIGPLAN International Haskell Symposium*, Haskell 2024, pages 120–124, New York, NY, USA, 2024. Association for Computing Machinery. doi:10.1145/3677999.3678280.
- 4 Jon Barwise. An introduction to first-order logic. In Jon Barwise, editor, *Handbook of Mathematical Logic*, volume 90 of *Studies in Logic and the Foundations of Mathematics*, pages 5–46. North-Holland, Amsterdam, 1977. doi:10.1016/S0049-237X(08)71097-8.
- 5 Eli Ben-Sasson, Alessandro Chiesa, Daniel Genkin, Eran Tromer, and Madars Virza. SNARKs for C: Verifying program executions succinctly and in zero knowledge. In Ran Canetti and Juan A. Garay, editors, *Advances in Cryptology – CRYPTO 2013*, volume 8043 of *Lecture Notes in Computer Science*, pages 90–108. Springer, Berlin, Heidelberg, 2013. doi:10.1007/978-3-642-40084-1_6.
- 6 Eli Ben-Sasson, Alessandro Chiesa, and Nicholas Spooner. Interactive oracle proofs. In Martin Hirt and Adam Smith, editors, *Theory of Cryptography*, volume 9986 of *Lecture Notes in Computer Science*, pages 31–60. Springer, Berlin, Heidelberg, 2016. doi:10.1007/978-3-662-53644-5_2.
- 7 Alexander R. Block, Zhiyong Fang, Jonathan Katz, Justin Thaler, Hendrik Waldner, and Yupeng Zhang. Field-agnostic SNARKs from expand-accumulate codes. In Leonid Reyzin and Douglas Stebila, editors, *Advances in Cryptology – CRYPTO 2024*, volume 14929 of *Lecture Notes in Computer Science*, pages 276–307, Cham, 2024. Springer Nature Switzerland. doi:10.1007/978-3-031-68403-6_9.
- 8 Benedikt Bünz, Ben Fisch, and Alan Szepieniec. Transparent SNARKs from DARK compilers. In Anne Canteaut and Yuval Ishai, editors, *Advances in Cryptology – EUROCRYPT 2020*, volume 12105 of *Lecture Notes in Computer Science*, pages 677–706. Springer, Cham, 2020. doi:10.1007/978-3-030-45721-1_24.
- 9 Matteo Campanelli and Mathias Hall-Andersen. Fully succinct arguments over the integers from first principles. Cryptology ePrint Archive, Paper 2024/1548, 2024. URL: <https://eprint.iacr.org/2024/1548>.
- 10 Matteo Campanelli and Mathias Hall-Andersen. General techniques for building SNARKs over the integers. In Shi Bai and Edoardo Persichetti, editors, *Public-Key Cryptography – PKC 2026*, volume 16553 of *Lecture Notes in Computer Science*, pages 197–230, Cham, 2026. Springer Nature Switzerland. doi:10.1007/978-3-032-26737-5_7.
- 11 T. J. W. Clarke, P. J. S. Gladstone, C. D. MacLean, and A. C. Norman. SKIM – the S, K, I reduction machine. In *Proceedings of the 1980 ACM Conference on LISP and Functional Programming*, LFP ’80, pages 128–135, New York, NY, USA, 1980. Association for Computing Machinery. doi:10.1145/800087.802798.
- 12 Benjamin E. Diamond and Jim Posen. Succinct arguments over towers of binary fields. In Serge Fehr and Pierre-Alain Fouque, editors, *Advances in Cryptology – EUROCRYPT 2025*, volume 15604 of *Lecture Notes in Computer Science*, pages 93–122, Cham, April 2025. Springer Nature Switzerland. doi:10.1007/978-3-031-91134-7_4.
- 13 William Ewald. The Emergence of First-Order Logic. In Edward N. Zalta, editor, *The Stanford Encyclopedia of Philosophy*. Metaphysics Research Lab, Stanford University, spring 2019 edition, 2019. URL: <https://plato.stanford.edu/archives/spr2019/entries/logic-firstorder-emergence/>.

- 14 Murdoch J. Gabbay. Arithmetisation of computation via polynomial semantics for first-order logic. Cryptology ePrint Archive, Paper 2024/954, 2024. URL: <https://eprint.iacr.org/2024/954>.
- 15 Murdoch J. Gabbay. Arithmetising logic: Polynomial semantics of FOL. *Journal of Applied Logics—IfCoLog Journal of Logics and their Applications*, 12(6):1479–1546, October 2025. URL: <https://www.collegepublications.co.uk/ifcolog/?00074>.
- 16 Ariel Gabizon, Zachary J. Williamson, and Oana Ciobotaru. PLONK: Permutations over Lagrange-bases for oecumenical noninteractive arguments of knowledge. Cryptology ePrint Archive, Paper 2019/953, 2019. URL: <https://eprint.iacr.org/2019/953>.
- 17 Chaya Ganesh, Anca Nitulescu, and Eduardo Soria-Vazquez. Rinocchio: SNARKs for ring arithmetic. *Journal of Cryptology*, 36(4):41, October 2023. doi:10.1007/s00145-023-09481-3.
- 18 Albert Garreta, Hendrik Waldner, Ilia Vlasov, Katerina Hristova, Luca Dall’Ava, Marko Čupić, and Matthew Klein. Zinc: Succinct arguments with small arithmetization overheads from IOPs of proximity to the integers. In Yael Tauman Kalai and Seny F. Kamara, editors, *Advances in Cryptology – CRYPTO 2025*, volume 16006 of *Lecture Notes in Computer Science*, pages 259–291. Springer, Cham, 2025. doi:10.1007/978-3-032-01907-3_9.
- 19 Alexander Golovnev, Jonathan Lee, Srinath Setty, Justin Thaler, and Riad S. Wahby. Brakedown: Linear-time and field-agnostic SNARKs for R1CS. In Helena Handschuh and Anna Lysyanskaya, editors, *Advances in Cryptology – CRYPTO 2023*, volume 14082 of *Lecture Notes in Computer Science*, pages 193–226, Cham, 2023. Springer Nature Switzerland. doi:10.1007/978-3-031-38545-2_7.
- 20 Jens Groth. On the size of pairing-based non-interactive arguments. In Marc Fischlin and Jean-Sébastien Coron, editors, *Advances in Cryptology – EUROCRYPT 2016*, volume 9666 of *Lecture Notes in Computer Science*, pages 305–326, Berlin, Heidelberg, April 2016. Springer Berlin Heidelberg. doi:10.1007/978-3-662-49896-5_11.
- 21 Kunming Jiang, Devora Chait-Roth, Zachary DeStefano, Michael Walfish, and Thomas Wies. Less is more: refinement proofs for probabilistic proofs. In *2023 IEEE Symposium on Security and Privacy (SP)*, pages 1112–1129. IEEE, 2023. doi:10.1109/SP46215.2023.10179393.
- 22 Junkai Liang, Daqi Hu, Pengfei Wu, Yunbo Yang, Qingni Shen, and Zhonghai Wu. SoK: Understanding zk-SNARKs: The gap between research and practice. In *34th USENIX Security Symposium (USENIX Security 25)*, pages 2085–2104, Seattle, WA, August 2025. USENIX Association. URL: <https://www.usenix.org/conference/usenixsecurity25/presentation/liang-sok>.
- 23 Jürgen Nicklisch-Franken and Ruslan Feizerakhmanov. Massimult: A novel parallel CPU architecture based on combinator reduction, 2024. arXiv:2412.02765, doi:10.48550/arXiv.2412.02765.
- 24 Anca Nitulescu. zk-SNARKs: A gentle introduction, 2020. URL: <https://www.di.ens.fr/~nitulesc/files/Survey-SNARKs.pdf>.
- 25 Michele Orrù, George Kadianakis, Mary Maller, and Greg Zaverucha. Beyond the circuit: How to minimize foreign arithmetic in ZKP circuits. *IACR Communications in Cryptology*, 2(1), April 2025. doi:10.62056/an-4c3c2h.
- 26 Bryan Parno, Jon Howell, Craig Gentry, and Mariana Raykova. Pinocchio: Nearly practical verifiable computation. In *2013 IEEE Symposium on Security and Privacy*, pages 238–252. IEEE, May 2013. doi:10.1109/SP.2013.47.
- 27 Srinath Setty, Justin Thaler, and Riad Wahby. Customizable constraint systems for succinct arguments. Cryptology ePrint Archive, Paper 2023/552, 2023. URL: <https://eprint.iacr.org/2023/552>.
- 28 Justin Thaler. Proofs, arguments, and Zero-Knowledge. *Foundations and Trends in Privacy and Security*, 4(2–4):117–660, December 2022. doi:10.1561/33000000030.
- 29 Urbit Systems Technical Journal. Nock 4K, 2018. Accessed 28 September 2026. URL: <https://nock.is/content/history/nock-4k/>.

- 30 Riad S. Wahby, Ioanna Tzialla, abhi shelat, Justin Thaler, and Michael Walfish. Doubly-efficient zkSNARKs without trusted setup. In *2018 IEEE Symposium on Security and Privacy (SP)*, pages 926–943. IEEE, May 2018. doi:10.1109/SP.2018.00060.
- 31 Yuanju Wei, Xinxuan Zhang, and Yi Deng. Transparent SNARKs over Galois rings. In Tibor Jager and Jiaxin Pan, editors, *Public-Key Cryptography – PKC 2025*, volume 15674 of *Lecture Notes in Computer Science*, pages 418–451, Cham, 2025. Springer Nature Switzerland. doi:10.1007/978-3-031-91820-9_14.
- 32 Zhuo Wu, Xinxuan Zhang, Yi Deng, Yuanju Wei, Zhongliang Zhang, and Liuyu Yang. Polylogarithmic polynomial commitment scheme over Galois rings. In Vincent Nicomette, Abdelmalek Benzekri, Nora Boulahia-Cuppens, and Jaideep Vaidya, editors, *Computer Security – ESORICS 2025*, volume 16054 of *Lecture Notes in Computer Science*, pages 400–420, Cham, 2026. Springer Nature Switzerland. doi:10.1007/978-3-032-07891-9_21.
- 33 Hadas Zeilberger, Binyi Chen, and Ben Fisch. BaseFold: Efficient field-agnostic polynomial commitment schemes from foldable codes. In Leonid Reyzin and Douglas Stebila, editors, *Advances in Cryptology – CRYPTO 2024*, volume 14929 of *Lecture Notes in Computer Science*, pages 138–169, Cham, 2024. Springer Nature Switzerland. doi:10.1007/978-3-031-68403-6_5.

A On Lookup Relations

Lookup relations

$$\text{Look}_{\text{gp}, \mathcal{R}} = \left\{ (\mathbf{i}, \mathbf{x}, \mathbf{w}) \left| \begin{array}{l} \text{gp} = (1, 0, 1, \log n_a + \log n_t, \partial), \\ \mathbf{i} = ([t]), \\ \mathbf{x} = ([a]), \\ \mathbf{w} = (a(\mathbf{X})), \\ a(\mathbf{X}) \in \mathcal{R}_{\partial}^{\#}[\mathbf{X}], \mathbf{X} = (X_1, \dots, X_{\log n_a}), \\ t(\mathbf{Y}) \in \mathcal{R}_{\partial}^{\#}[\mathbf{Y}], \mathbf{Y} = (Y_1, \dots, Y_{\log n_t}), \\ \{a(\mathbf{x}) \mid \mathbf{x} \in \{0, 1\}^{\log n_a}\} \subseteq \{t(\mathbf{y}) \mid \mathbf{y} \in \{0, 1\}^{\log n_t}\} \end{array} \right. \right\}$$

are defined as AIRs as follows:

Let $a(\mathbf{X})$ and $t(\mathbf{Y})$ be multilinear polynomials on $\log n_a$ and $\log n_t$ variables respectively, and let $\mathbf{a}$ and $\mathbf{t}$ be the vectors of evaluations of $a(\mathbf{X})$ and $t(\mathbf{Y})$ on the hypercubes $\mathbf{a} = (a(\mathbf{X}))_{\{0,1\}^{\log n_a}}$ and $\mathbf{t} = (t(\mathbf{Y}))_{\{0,1\}^{\log n_t}}$, respectively. Define variables

$$\mathbf{W} = ((W_{\mathbf{x}})_{\mathbf{x} \in \{0,1\}^{\log n_a}}, (W_{\mathbf{y}})_{\mathbf{y} \in \{0,1\}^{\log n_t}})$$

and polynomials

$$Q_{\text{Look}} = \left\{ Q_{\mathbf{x}}(\mathbf{W}) := \prod_{\mathbf{y} \in \{0,1\}^{\log n_t}} (W_{\mathbf{x}} - W_{\mathbf{y}}) \mid \mathbf{x} \in \{0,1\}^{\log n_a} \right\}$$

Then

$$\text{REL}_{\text{gp}, \mathcal{R}, Q_{\text{Look}}} = \left\{ (\mathbf{i}, \mathbf{x}, \mathbf{w}) \left| \begin{array}{l} \text{gp} = (1, 0, 1, \log n_a + \log n_t, \partial), \\ \mathbf{i} = ([t]), \quad \mathbf{x} = ([a]), \\ a(\mathbf{X}) \in \mathcal{R}_{\partial}^{\#}[\mathbf{X}], \mathbf{X} = (X_1, \dots, X_{\log n_a}), \\ t(\mathbf{Y}) \in \mathcal{R}_{\partial}^{\#}[\mathbf{Y}], \mathbf{Y} = (Y_1, \dots, Y_{\log n_t}), \\ \mathbf{w} = (a(\mathbf{X})), \\ Q_{\mathbf{x}}(\mathbf{a}, \mathbf{t}) = 0, \text{ for all } \mathbf{x} \in \{0,1\}^{\log n_a} \end{array} \right. \right\} \quad (2)$$

is equivalent to the (standard) lookup relation Look_{gp} [18, Section 4.4].

B Proof of Theorem 19

Proof. We reason as follows ([15, Theorem 2.4.4] has an analogous proof):

1. It is a fact that $0 = 0$ and $1 \neq 0$.
2. It is a fact that $x = x'$ if and only if $(x - x')^2 = 0$.
3. By non-negativity (Lemma 17).
4. The product of two non-negative numbers is non-zero if and only if they both are.
5. It is a fact that $0 = 0$.
6. By construction: range checks compile to 0 and 1 (depending on values of the indicator function δ). Their sum is zero if and only if all summands are zero.
7. As for the previous item. ◀

C Proof of Corollary 32

For the reader's benefit, we recall the meaning of each parameter contained in the global parameters $\text{gp} = (k, m, n, \mu, \partial)$: k is the number of multilinear polynomials f_i comprising the

witness; m is the rank of the auxiliary data $\mathbf{y}$ in the instance; n is the number of indexing multilinear polynomials g_i in $\mathbf{i}$; μ is the dimension of the hypercube onto which the f_i and g_i interpolate; and ∂ is the bit-length bound on the coefficients of the instance-witness data.

Proof. As in Lemma 30 we transform the instance $((\phi, R), \varsigma) \in \text{REL}_{\text{gp}}^{\text{EP}}$ into an equivalent instance $(\mathbf{x}, \mathbf{w}) \in \text{REL}_{\text{gp}, \mathcal{Q}}$ which is a valid input to Zinc-PIOP, for some parameters $\text{gp} = (k, m, n, \mu, \partial)$. The claim concerning the soundness error follows from running a SNARK compiled from Zinc-PIOP and Zip. To conclude, it suffices to compute gp .

Let $((mkQ_{\mathcal{F}}^x(\phi))_{x \in [\text{len}(\varsigma(\mathbf{C}))]}, R), (\tilde{f}_{\varsigma(\mathbf{C})}^i)_{i \in [\text{ar}(\mathbf{C})]}) \in \text{REL}_{\text{gp}, \mathcal{F}}^{\text{MV}}$ be obtained from the FOL instance $((\phi, R), \varsigma)$. Following the proof of Lemma 30, we convert these data into an AIR for Zinc. We obtain one instance-witness pair by considering $(mkQ_{\mathcal{F}}^x(\phi))_{x \in [\text{len}(\varsigma(\mathbf{C}))]}$ and another by considering the range checks R ; by relabelling indices and defining $\mathcal{Q}$, $\mathbf{i}$, $\mathbf{x}$, and $\mathbf{w}$ appropriately we can combine these two instances into a single Zinc input relation tuple. Here, however, we (equivalently) consider the two relations side by side, for clarity.

First consider $\text{REL}_{\text{gp}', \mathcal{Q}'}$ where

$$\mathcal{Q}' := ((mkQ_{\mathcal{F}}^x(\phi))_{x \in [\text{len}(\varsigma(\mathbf{C}))]}, x_1(x_1 - 1), \dots, x_\mu(x_\mu - 1)),$$

and $\mathbf{x} = ([\tilde{f}_{\varsigma(\mathbf{C})}^1], \dots, [\tilde{f}_{\varsigma(\mathbf{C})}^{\text{ar}(\mathbf{C})}])$, and $\mathbf{w} = (\tilde{f}_{\varsigma(\mathbf{C})}^1, \dots, \tilde{f}_{\varsigma(\mathbf{C})}^{\text{ar}(\mathbf{C})})$. Here

$$\text{gp}' = (\text{ar}(\mathbf{C}), 0, 0, \mu, \partial)$$

Next, consider the instances of $\text{REL}_{\text{gp}'', \mathcal{Q}_{\text{Look}}}$ (cf. equation (2)) corresponding to R ; any single range check $R_i \in R$ has global parameters

$$\text{gp}_i'' = (1, 0, 1, \log n_a + \log n_t, \partial)$$

for some n_a, n_t . We next compute $\log n_a$ and $\log n_t$.

Recall $\log n_a$ is the number of variables on which the multilinear extension comprising the range check witness $\mathbf{w} = a(\mathbf{X})$ is defined. Since range checks in R are range checks on the rows of $\varsigma(\mathbf{C})$, we may take $a(\mathbf{X}) = \tilde{f}_{\varsigma(\mathbf{C})}^i$ for some $i \in [\text{ar}(\mathbf{C})]$ and so each range check in R comprises at most $\text{len}(\varsigma(\mathbf{C}))$ integer range checks. Then n_a may be taken to be the minimal integer satisfying $\lceil \log n_a \rceil = \text{maxbl} = \mu$. By padding appropriately we take $\log n_a = \mu$.

Recall $t(\mathbf{Y})$ is the multilinear extension interpolating the set in which the evaluations of $a(\mathbf{X})$ should lie. Since range checks in our semantics arise to ensure pointers are well-defined, range checks ensure that the entries of rows of $\varsigma(\mathbf{C})$ lie in $[\text{len}(\varsigma(\mathbf{C}))]$. Thus $\log n_t = \mu$ also.

Finally, since R may contain many range checks, we find that the parameters to perform all the range checks in R are

$$\text{gp}'' = (|R|, 0, 1, (|R| + 1)\mu, \partial)$$

To compute the final parameters $\text{gp} = (k, m, n, \mu, \partial)$ of our AIR instance, we note that since the range checks are performed on the same witness data as in the $\text{REL}_{\text{gp}', \mathcal{Q}'}$ instance, we have $k = \text{ar}(\mathbf{C})$. Moreover, the same choice of $t(\mathbf{Y})$ can be used for each range check, so $n = 1$. Thus the final parameters of the AIR instance are

$$\text{gp} = (\text{ar}(\mathbf{C}), 0, 1, (|R| + 2)\mu, \partial)$$

◀

D

 Two Examples of Arithmetisation in Action

Here we provide concrete examples of sample FOL predicates mapping to instance-witness input pairs for our SNARK. These examples are selected for their illustrative value, not for displaying the compactness nor the efficiency of our transform from FOL to SNARK inputs. The latter task of optimising our transform is primarily an *engineering* challenge, whereas here we are concerned with the theoretical and conceptual aspects of our work.

Arithmetising power functions A common use of SNARKs sees a prover convince a verifier that the prover has computed the output of some function. We now show how a prover can prove knowledge of values of a function expressed in FOL. We consider power functions:

► **Definition 35.** *The standard inductive definition of a^b for $a, b \in \mathbb{N}$ is a function $\text{pow}(x, y)$ satisfying:*

$$\begin{array}{ll} \text{base case} & \text{pow}(a, 0) = 1 \\ \text{inductive step} & \text{pow}(a, b + 1) = a * \text{pow}(a, b) \end{array}$$

This may be written in first-order logic as

$$\forall a. (\text{pow}(a, 0) = 1) \wedge \forall a, b. (\text{pow}(a, b + 1) = a * \text{pow}(a, b)),$$

We define the corresponding predicate expressed via enriched polynomials below, where ϕ_{pow} is the validity predicate giving the condition that pow is an arity 4 matrix variable symbol with rows pow_i which encode the power function, and where $\text{pow}_i(X)$ denotes an enriched polynomial. We let the entries of pow_1 correspond to bases, pow_2 correspond to exponents, pow_3 correspond to the function output, and pow_4 correspond to pointers. Set

$$\begin{aligned} \text{baseCase}(X) &= (\text{pow}_2(X) = 0) \wedge (\text{pow}_3(X) = 1), \\ \text{indStep}(X) &= \text{pow}_1(X) = \text{pow}_1(\text{pow}_4(X)) \quad \wedge \\ &\quad \text{pow}_2(X) = \text{pow}_2(\text{pow}_4(X)) + 1 \quad \wedge \\ &\quad \text{pow}_3(X) = \text{pow}_1(X) \cdot \text{pow}_3(\text{pow}_4(X)) \end{aligned}$$

Then — using a chained- $\leq$ shorthand for range checks — set

$$(\phi_{\text{pow}}, R_{\text{pow}}) = (\text{baseCase}(X) \vee \text{indStep}(X), 1 \leq \text{pow}_4 \leq \text{len}(\text{pow}))$$

See [15, Lemma 3.2.6] for a proof that $(\phi_{\text{pow}}, R_{\text{pow}})$ does indeed encode Definition 35. We therefore consider instances of the relation

$$\text{Rel}_{\text{gp}}^{\text{FOL}} = \left\{ ((\phi, R), \varsigma) \mid \begin{array}{l} \phi = \phi_{\text{pow}}, R = R_{\text{pow}}, \mathbf{C} = \text{pow} \\ \varsigma(\mathbf{C}) \in \mathcal{M}_{4 \times \text{len}(\varsigma(\mathbf{C}))}(\mathbb{N}), \quad \mathbf{C} \models_{\varsigma} \phi; R \end{array} \right\}$$

Applying the transform of Figure 2 to ϕ_{pow} , we obtain an enriched polynomial instance in

$$\text{Rel}_{\text{gp}}^{\text{EP}} = \left\{ ((\langle \phi \rangle, R), \varsigma) \mid \begin{array}{l} \phi = \phi_{\text{pow}}, R = R_{\text{pow}}, \mathbf{C} = \text{pow} \\ \varsigma(\mathbf{C}) \in \mathcal{M}_{4 \times \text{len}(\varsigma(\mathbf{C}))}(\mathbb{N}), \quad \mathbf{C} \models_{\varsigma} \phi; R \end{array} \right\}$$

$$\begin{aligned} \text{where } \langle \phi_{\text{pow}} \rangle &= (\text{pow}_2(X)^2 + (\text{pow}_3(X) - 1)^2) * \\ &\quad ((\text{pow}_1(X) - \text{pow}_1(\text{pow}_4(X)))^2 + (\text{pow}_2(X) - (\text{pow}_2(\text{pow}_4(X)) + 1))^2 \\ &\quad + (\text{pow}_3(X) - \text{pow}_1(X) * \text{pow}_3(\text{pow}_4(X)))^2) \end{aligned}$$

We then apply $mkQ_{\mathcal{F}}^x$ with $\mathcal{F} = \{(\tilde{f}_{\varsigma(\text{pow})}^i)_{i \in [4]}\}$ to obtain an instance in

$$\text{REL}_{\text{gp}, \mathcal{F}}^{\text{MV}} = \left\{ \left(((mkQ_{\mathcal{F}}^x(\phi))_{x \in [\text{len}(\varsigma(\mathbb{C}))]}, R), (\tilde{f}_{\varsigma(\mathbb{C})}^i)_{i \in [4]} \right) \left| \begin{array}{l} \phi = \phi_{\text{pow}}, R = R_{\text{pow}}, \mathbb{C} = \text{pow} \\ \varsigma(\mathbb{C}) \in \mathcal{M}_{4 \times \text{len}(\varsigma(\mathbb{C}))}(\mathbb{N}), \mathbb{C} \models_{\varsigma} R \\ \forall x \in [\text{len}(\varsigma(\mathbb{C}))] (\beta_{\mathcal{F}}(mkQ_{\mathcal{F}}^x(\phi)) = 0) \end{array} \right. \right\}$$

By Figure 4 the instance polynomials have the form

$$\begin{aligned} mkQ_{\mathcal{F}}^x(\phi_{\text{pow}}) &= (mkQ_{\mathcal{F}}^x(\text{pow}_2(X))^2 + (mkQ_{\mathcal{F}}^x(\text{pow}_3(X)) - 1)^2) * \\ &\quad \left((mkQ_{\mathcal{F}}^x(\text{pow}_1(X)) - mkQ_{\mathcal{F}}^x(\text{pow}_1(\text{pow}_4(X))))^2 \right. \\ &\quad \left. + (mkQ_{\mathcal{F}}^x(\text{pow}_2(X)) - (mkQ_{\mathcal{F}}^x(\text{pow}_2(\text{pow}_4(X))) + 1))^2 \right. \\ &\quad \left. + (mkQ_{\mathcal{F}}^x(\text{pow}_3(X)) - mkQ_{\mathcal{F}}^x(\text{pow}_1(X)) * mkQ_{\mathcal{F}}^x(\text{pow}_3(\text{pow}_4(X))))^2 \right) \\ &= (b2int(B_{2,1}^{0,x}, \dots, B_{2,\mu}^{0,x})^2 + (b2int(B_{3,1}^{0,x}, \dots, B_{3,\mu}^{0,x}) - 1)^2) * \\ &\quad \left((b2int(B_{1,1}^{0,x}, \dots, B_{1,\mu}^{0,x}) - b2int(B_{1,1}^{4,x}, \dots, B_{1,\mu}^{4,x}))^2 \right. \\ &\quad \left. + (b2int(B_{2,1}^{0,x}, \dots, B_{2,\mu}^{0,x}) - (b2int(B_{2,1}^{4,x}, \dots, B_{2,\mu}^{4,x}) + 1))^2 \right. \\ &\quad \left. + (b2int(B_{3,1}^{0,x}, \dots, B_{3,\mu}^{0,x}) - b2int(B_{1,1}^{0,x}, \dots, B_{1,\mu}^{0,x}) * b2int(B_{3,1}^{4,x}, \dots, B_{3,\mu}^{4,x}))^2 \right) \end{aligned}$$

The condition $\beta_{\mathcal{F}}(mkQ_{\mathcal{F}}^x(\phi_{\text{pow}})) = 0$ is equivalent to saying that when $\beta_{\mathcal{F}}$ replaces the $B_{i,\nu}^{0,x}$ with $\pi_{\nu} \mathfrak{D}_{\mathfrak{b}} \tilde{f}_{\varsigma(\text{pow})}^i(\mathfrak{D}_{\mathfrak{b}} x)$ and the $B_{i,\nu}^{4,x}$ with $\pi_{\nu} \mathfrak{D}_{\mathfrak{b}} \tilde{f}_{\varsigma(\text{pow})}^i(\mathfrak{D}_{\mathfrak{b}} \tilde{f}_{\varsigma(\text{pow})}^4(\mathfrak{D}_{\mathfrak{b}} x))$, that is when $\beta_{\mathcal{F}}$ replaces the $B_{i,\nu}^{0,x}$ with $\pi_{\nu} \mathfrak{D}_{\mathfrak{b}} \varsigma(\text{pow})_{@i,x}$ and the $B_{i,\nu}^{4,x}$ with $\pi_{\nu} \mathfrak{D}_{\mathfrak{b}} \varsigma(\text{pow})_{@i, \varsigma(\text{pow})_{@4,x}}$, we obtain 0. Thus $mkQ_{\mathcal{F}}^x(\phi_{\text{pow}})$ has roots at the binary decompositions of (a subset of) the entries of $\varsigma(\text{pow})$, if ς is a valid interpretation.

The final $\text{REL}_{\text{gp}, \mathcal{Q}}$ instance, ignoring range checks for simplicity, has polynomial constraints $\mathcal{Q} = (mkQ_{\mathcal{F}}^x(\phi_{\text{pow}}))_{x \in [\text{len}(\varsigma(\text{pow}))]}$ and witness $\mathbb{w} = (\tilde{f}_{\varsigma(\mathbb{C})}^i)_{i \in [4]}$.

Arithmetising SK Combinator reduction We next consider SNARKs for SK combinator reduction, a Turing-complete system of computation. This allows us to take as an instance any Turing-complete computation, compiled to SK combinator calculus, and run a SNARK on the statement expressed in our FOL semantics to allow a prover to prove knowledge of witnesses to properties of such computations.

The interested reader can find more on SK combinator reduction in [15, Section 3.9]. A nice summary of the benefits of combinator reduction, and how it relates to other computational models, is contained in the early pages of [23]; see also [11]. Furthermore a SNARK-friendly treatment of SK combinators is the proposed *EDEN* system [1].¹²

We begin by defining the Cantor pairing function:

¹²To our knowledge [1] is not a peer-reviewed publication. However, if it is correct it is a developed and practical SNARK-friendly treatment of SK combinators (represented as Dyck words injected into strings of finite field elements). We quote the authors on why this matters: “Most zkVMs emulate the von Neumann architecture and must prove relations between a program’s execution and its use of Random Access Memory. However, there are conceptually simpler models of computation that are naturally modelled in a SNARK yet are still practical for use.” Amen. With respect to that paper, our ‘zkVM’ is FOL itself, and in this section we show a simple but direct and effective way to encode SK combinator reduction within it.

► **Definition 36.** Define a *Cantor pairing map*¹³

$$\begin{aligned} \langle \cdot, \cdot \rangle' : \mathbb{N} \times \mathbb{N} &\rightarrow \mathbb{N} \\ \langle x, y \rangle' &= (x + y) * (x + y + 1) + 2 * x \end{aligned}$$

We now arithmetise SK combinator reduction. We first define a relation, where ϕ_{SK} is a validity predicate and SK is a matrix variable symbol encoding combinator reduction:

$$\text{Rel}_{\text{gp}}^{\text{FOL}} = \left\{ ((\phi, R), \varsigma) \mid \begin{array}{l} \phi = \phi_{\text{SK}}, R = R_{\text{SK}}, \mathbf{C} = \text{SK} \\ \varsigma(\mathbf{C}) \in \mathcal{M}_{\text{ar}(\mathbf{C}) \times \text{len}(\varsigma(\mathbf{C}))}(\mathbb{N}), \quad \mathbf{C} \models_{\varsigma} \phi; R \end{array} \right\}$$

where SK has arity 8 with rows SK_i and ϕ_{SK} is defined as follows, where $\text{SK}_i(X)$ denotes an enriched polynomial and $\langle \cdot, \cdot \rangle := \langle \cdot, \cdot \rangle' + 2$, where $\langle \cdot, \cdot \rangle'$ is Cantor pairing. Set

$$\begin{aligned} \text{Kred}(X) &= \text{SK}_1(X) = \langle \langle 1, \text{SK}_2(X) \rangle, \text{SK}_4(X) \rangle, \\ \text{Sred}(X) &= \text{SK}_1(X) = \langle \langle \langle 0, \text{SK}_3(X) \rangle, \text{SK}_4(X) \rangle, \text{SK}_5(X) \rangle \quad \wedge \\ &\quad \text{SK}_2(X) = \langle \langle \text{SK}_3(X), \text{SK}_5(X) \rangle, \langle \text{SK}_4(X), \text{SK}_5(X) \rangle \rangle, \\ \text{Id}(X) &= \text{SK}_1(X) = \text{SK}_2(X), \\ \text{Par}(X) &= \text{SK}_1(X) = \langle \text{SK}_1(\text{SK}_6(X)), \text{SK}_1(\text{SK}_7(X)) \rangle \quad \wedge \\ &\quad \text{SK}_2(X) = \langle \text{SK}_2(\text{SK}_6(X)), \text{SK}_2(\text{SK}_7(X)) \rangle \quad \wedge \\ &\quad \text{SK}_8(X) = \text{SK}_8(\text{SK}_6(X)) + 1 \quad \wedge \\ &\quad \text{SK}_8(X) = \text{SK}_8(\text{SK}_7(X)) + 1, \\ \text{Tran}(X) &= \text{SK}_1(X) = \text{SK}_1(\text{SK}_6(X)) \quad \wedge \\ &\quad \text{SK}_2(X) = \text{SK}_2(\text{SK}_7(X)) \quad \wedge \\ &\quad \text{SK}_2(\text{SK}_6(X)) = \text{SK}_1(\text{SK}_7(X)) \quad \wedge \\ &\quad \text{SK}_8(X) = \text{SK}_8(\text{SK}_6(X)) + 1 \quad \wedge \\ &\quad \text{SK}_8(X) = \text{SK}_8(\text{SK}_7(X)) + 1 \end{aligned}$$

Then set

$$\begin{aligned} (\phi_{\text{SK}}, R_{\text{SK}}) &= (\text{Kred}(X) \vee \text{Sred}(X) \vee \text{Id}(X) \vee \text{Par}(X) \vee \text{Tran}(X), \\ &\quad 1 \leq \text{SK}_6 \leq \text{len}(\text{SK}) \wedge 1 \leq \text{SK}_7 \leq \text{len}(\text{SK})) \end{aligned}$$

See [15, Lemma 3.9.10] for a proof that $(\phi_{\text{SK}}, R_{\text{SK}})$ does encode SK combinator reduction. Applying the transform of Figure 2 to ϕ_{SK} , we obtain an enriched polynomial instance in

$$\text{Rel}_{\text{gp}}^{\text{EP}} = \left\{ (((\phi), R), \varsigma) \mid \begin{array}{l} \phi = \phi_{\text{SK}}, R = R_{\text{SK}}, \mathbf{C} = \text{SK} \\ \varsigma(\mathbf{C}) \in \mathcal{M}_{8 \times \text{len}(\varsigma(\mathbf{C}))}(\mathbb{N}), \quad \mathbf{C} \models_{\varsigma} \phi; R \end{array} \right\}$$

where R_{SK} contains the range checks $1 \leq \text{SK}_6 \leq \text{len}(\text{SK})$ and $1 \leq \text{SK}_7 \leq \text{len}(\text{SK})$, and

$$\begin{aligned} \langle \phi_{\text{SK}} \rangle &= (\text{SK}_1(X) - \langle \langle 1, \text{SK}_2(X) \rangle, \text{SK}_4(X) \rangle)^2 \\ &\quad * (\text{SK}_1(X) - \langle \langle \langle 0, \text{SK}_3(X) \rangle, \text{SK}_4(X) \rangle, \text{SK}_5(X) \rangle \rangle)^2 \\ &\quad + (\text{SK}_2(X) - \langle \langle \text{SK}_3(X), \text{SK}_5(X) \rangle, \langle \text{SK}_4(X), \text{SK}_5(X) \rangle \rangle)^2 \end{aligned}$$

¹³The Cantor pairing function *bijects* $\mathbb{N} \times \mathbb{N}$ with $\mathbb{N}$ and is equal to $\langle x, y \rangle' / 2$. But this would require us to include fractions in our term language, which is a wrinkle which we prefer to avoid for simplicity, since here we just need some *injection* that can be expressed as a polynomial.

$$\begin{aligned}
& * (\text{SK}_1(X) - \text{SK}_2(X))^2 \\
& * ((\text{SK}_1(X) - \langle \text{SK}_1(\text{SK}_6(X)), \text{SK}_1(\text{SK}_7(X)) \rangle)^2 \\
& \quad + (\text{SK}_2(X) - \langle \text{SK}_2(\text{SK}_6(X)), \text{SK}_2(\text{SK}_7(X)) \rangle)^2 \\
& \quad + (\text{SK}_8(X) - \text{SK}_8(\text{SK}_6(X)) - 1)^2 + (\text{SK}_8(X) - \text{SK}_8(\text{SK}_7(X)) - 1)^2) \\
& * ((\text{SK}_1(X) - \text{SK}_1(\text{SK}_6(X)))^2 + (\text{SK}_2(X) - \text{SK}_2(\text{SK}_7(X)))^2 \\
& \quad + (\text{SK}_2(\text{SK}_6(X)) - \text{SK}_1(\text{SK}_7(X)))^2 + (\text{SK}_8(X) - \text{SK}_8(\text{SK}_6(X)) - 1)^2 \\
& \quad + (\text{SK}_8(X) - \text{SK}_8(\text{SK}_7(X)) - 1)^2)
\end{aligned}$$

using the transform of Figure 2.

Applying $mkQ_{\mathcal{F}}^x$ to $\langle \phi_{\text{SK}} \rangle$ we obtain the $\text{len}(\varsigma(\text{SK}))$ polynomials that (partly) comprise the set $\mathcal{Q}$ for the instance of the relation upon which Zinc-PIOP may be run.